MOVIE RECOMMENDATION SYSTEM

A Project Report

In the partial fulfillment of the award of the degree of

BCA

at

ARDENT COMPUTECH PVT. LTD

Submitted by
SHUVAM MANDAL(Team Leader),
HARSH RAI,
PRIYANGSU MANDAL

Certificate from the Mentor

This is to certify that **SHUVAM MANDAL(**Team Leader), HARSH RAI,PRIYANGSU MANDAL has completed the project titled **MOVIE RECOMMENDATION SYSTEM** under my supervision during the period from **April** to **May** which is in partial fulfillment of requirements for the award of the **BCA** and submitted to Department **Computer Application** of **Swami** Vivekananda University.

_____

Signature of the Mentor

Date:

Acknowledgment

We take this opportunity to express our deep gratitude and sincere thanks to our project mentor, **Mr. Kunal Shit**, for his valuable suggestions, helpful guidance, and constant encouragement during the execution of this project work.

We would also like to give a special mention to our team members, as this project was completed through our combined efforts, cooperation, and mutual support.

Last but not least, we are grateful to all the faculty members of **Ardent Computech Pvt. Ltd.** for their continuous support and guidance.

ABSTRACT

The Movie Recommendation System is a Machine Learning based project that helps users discover movies according to their interests and preferences. The system analyzes movie data and recommends similar movies based on content and user preferences.

This project uses Python programming language along with Machine Learning libraries such as Pandas and Scikit-learn. The recommendation process is based on similarity techniques and data filtering methods.

The main objective of this project is to improve user experience by suggesting relevant and personalized movies quickly and efficiently.

TABLE OF CONTENTS

- Introduction
- Objective of the Project
- Scope of the Project
- Problem Statement
- Literature Review
- System Requirements
- Technology Used
- System Design
- Methodology
- Implementation
- Testing
- Output Screens
- Advantages and Limitations
- Future Scope
- Conclusion
- References

CHAPTER 1: INTRODUCTION

In today's digital world, online streaming platforms contain thousands of movies. Users often face difficulty in selecting movies according to their interests. A recommendation system helps users by suggesting movies based on their preferences.

The Movie Recommendation System uses Machine Learning algorithms to analyze movie information and user interests. It recommends movies that are similar in genre, cast, keywords, and ratings.

Recommendation systems are widely used in modern applications such as streaming platforms, e-commerce websites, and social media platforms.

This project demonstrates how Machine Learning techniques can improve user experience and automate movie suggestions.

❖❖❖❖❖ **End of Chapter** ❖❖❖❖❖

➢ CHAPTER 2: Objective of the Project

The main objectives of this project are:

- To develop a movie recommendation system using Machine Learning.
- To recommend movies based on user interests.
- To improve user experience.
- To understand data processing and recommendation algorithms.
- To implement Machine Learning concepts in a real-world application.

❖❖❖❖❖ **End of Chapter** ❖❖❖❖❖

CHAPTER 3: SCOPE OF THE PROJECT

The scope of this project includes:

- Recommending movies automatically.
- Reducing user effort in searching movies.
- Providing personalized movie suggestions.
- Learning practical implementation of Machine Learning.

This project can be further expanded into a web application or mobile application.

❖❖❖❖❖ **End of Chapter** ❖❖❖❖❖

CHAPTER 4: PROBLEM STATEMENT

With the rapid growth of digital entertainment platforms, users are often confused about which movie to watch. Searching manually for movies takes time and effort. The problem is to develop a system that can automatically recommend movies based on similarity and user preferences.
The system should provide fast and accurate recommendations.

❖❖❖❖❖ **End of Chapter** ❖❖❖❖❖

CHAPTER 5: LITERATURE REVIEW

Recommendation systems are important in modern online platforms. Many companies like Netflix and Amazon use recommendation systems to improve customer engagement.
There are mainly three types of recommendation systems:

1. Content-Based Filtering
2. Collaborative Filtering
3. Hybrid Recommendation System

This project uses a Content-Based Filtering approach where movies are recommended based on movie features and similarities.

❖❖❖❖❖ **End of Chapter** ❖❖❖❖❖

CHAPTER 6: SYSTEM REQUIREMENTS

Hardware Requirements

- Processor: Intel i3 or higher
- RAM: 4 GB minimum
- Hard Disk: 20 GB free space

Software Requirements

- Operating System: Windows 10/11
- Python 3.x
- Jupyter Notebook
- VS Code

❖❖❖❖❖ **End of Chapter** ❖❖❖❖❖

CHAPTER 7: TECHNOLOGY USED

Programming Language

- Python

Libraries Used

- Pandas
- NumPy
- Scikit-learn
- NLTK

Tools Used

- Jupyter Notebook
- VS Code

Python

Python is a high-level programming language used for Machine Learning and data analysis.

Pandas

Pandas is used for handling and analyzing datasets.

Scikit-learn

Scikit-learn is used for implementing Machine Learning algorithms

❖❖❖❖❖ End of Chapter ❖❖❖❖❖

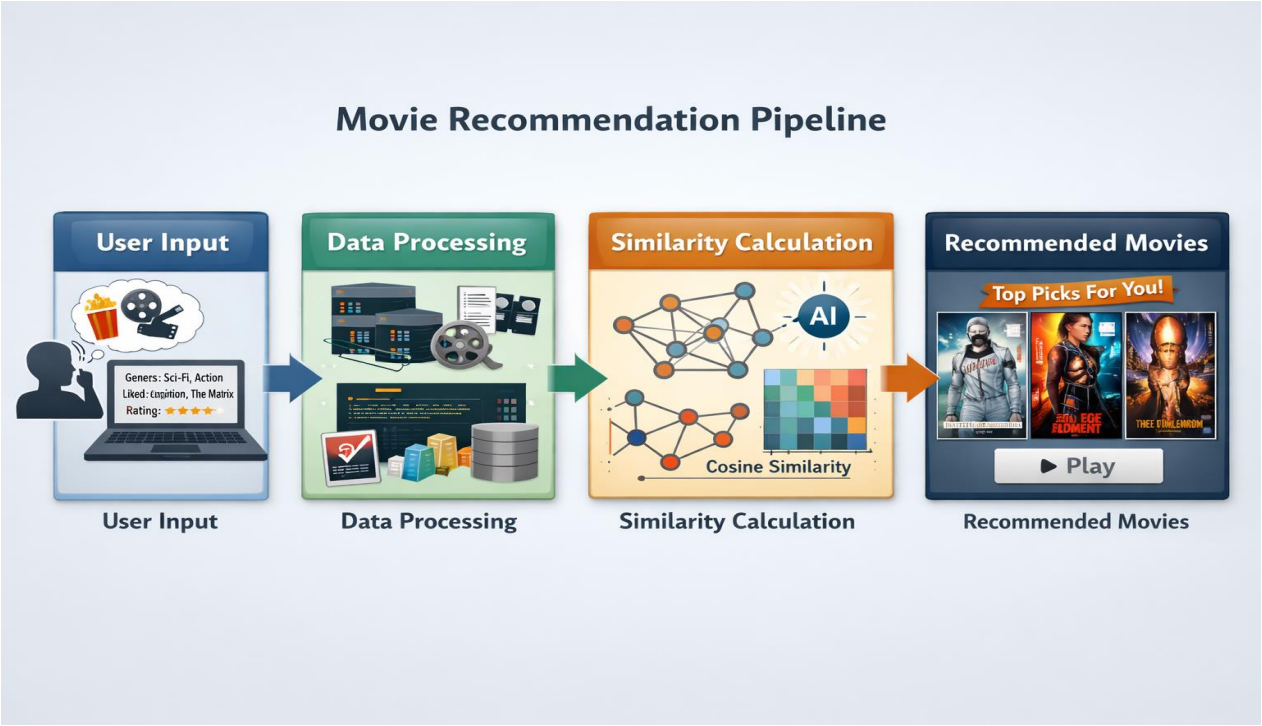
CHAPTER 8: SYSTEM DESIGN

Flow of the System

1. Collect Movie Dataset
2. Preprocess Data
3. Extract Important Features
4. Calculate Similarity
5. Recommend Similar Movies

Data Flow Diagram

User Input → Data Processing → Similarity Calculation → Recommended Movies



❖❖❖❖❖ End of Chapter ❖❖❖❖❖

CHAPTER 9: METHODOLOGY

The project follows the following methodology:

Step 1: Data Collection

A movie dataset is collected containing movie titles, genres, cast, keywords, and ratings.

Step 2: Data Preprocessing

Unnecessary data is removed and missing values are handled.

Step 3: Feature Extraction

Important features such as genres and keywords are extracted.

Step 4: Similarity Calculation

Cosine similarity technique is used to find similar movies.

Step 5: Recommendation

The system recommends top similar movies to the user.

❖❖❖❖❖ End of Chapter ❖❖❖❖❖

CHAPTER 10: IMPLEMENTATION

Sample Python Code

```
import streamlit as st
import pickle
import pandas as pd
import requests

def fetch_poster(movie_id):
    response =
requests.get("https://api.themoviedb.org/3/movie/{}?api_key=8265bd1679663a7ea12ac168da84d2e8&langu
age=en-US".format(movie_id))
    data = response.json()
    return "https://image.tmdb.org/t/p/w500/" + data['poster_path']

def recommend(movie):
    movie_index = movies[movies['title'] ==movie].index[0]
    distances = similarity[movie_index]
    movies_list = sorted(list(enumerate(distances)),reverse=True,key=lambda x:x[1])[1:6]

    recommended_movies = []
    recommended_movies_posters = []
    for i in movies_list:
        movie_id = movies.iloc[i[0]].movie_id

        recommended_movies.append(movies.iloc[i[0]].title)
        # fetch poster from api
        recommended_movies_posters.append(fetch_poster(movie_id))
    return recommended_movies,recommended_movies_posters


movies_dict = pickle.load(open("movie_dict.pkl", "rb"))
movies = pd.DataFrame(movies_dict)

similarity = pickle.load(open("similarity.pkl", "rb"))


st.title('Movie recommender system')
selected_movie_name = st.selectbox(
    'How would you like to be contacted?',
    movies['title'].values
)


if st.button('Recommend'):
   names,posters = recommend(selected_movie_name)

   col1, col2, col3, col4, col5 = st.columns(5)
   with col1:
      st.text(names[0])
      st.image(posters[0])
```

```
with col2:
    st.text(names[1])
    st.image(posters[1])

with col3:
    st.text(names[2])
    st.image(posters[2])
with col4:
    st.text(names[3])
    st.image(posters[3])
with col5:
    st.text(names[4])
    st.image(posters[4])
```

Explanation of the Code

- Pandas is used to load the dataset.
- CountVectorizer converts text into numerical vectors.
- Cosine similarity calculates similarity between movies.
- The system recommends similar movies.

❖❖❖❖❖ **End of Chapter** ❖❖❖❖❖

CHAPTER 10: FUTURE SCOPE

CHAPTER 11: TESTING

Testing is important to ensure that the system works correctly.

Types of Testing

Unit Testing

Individual modules are tested separately.

System Testing

The complete system is tested together.

Performance Testing

Checks the speed and efficiency of recommendations.

SEMPLE PYTHON CODE

```
import abc
import builtins
import collections
import collections.abc
import contextlib
import enum
```

```
import functools
import inspect
import io
import keyword
import operator
import sys
import types as _types
import typing
import warnings

# Breakpoint: https://github.com/python/cpython/pull/119891
if sys.version_info >= (3, 14):
    import annotationlib

__all__ = [
    # Super-special typing primitives.
    'Any',
    'ClassVar',
    'Concatenate',
    'Final',
    'LiteralString',
    'ParamSpec',
    'ParamSpecArgs',
    'ParamSpecKwargs',
    'Self',
    'Type',
    'TypeVar',
    'TypeVarTuple',
    'Unpack',

    # ABCs (from collections.abc).
    'Awaitable',
    'AsyncIterator',
    'AsyncIterable',
    'Coroutine',
    'AsyncGenerator',
    'AsyncContextManager',
    'Buffer',
    'ChainMap',

    # Concrete collection types.
    'ContextManager',
    'Counter',
    'Deque',
```

'DefaultDict',
'NamedTuple',
'OrderedDict',
'TypedDict',

Structural checks, a.k.a. protocols.
'SupportsAbs',
'SupportsBytes',
'SupportsComplex',
'SupportsFloat',
'SupportsIndex',
'SupportsInt',
'SupportsRound',
'Reader',
'Writer',

One-off things.
'Annotated',
'assert_never',
'assert_type',
'clear_overloads',
'dataclass_transform',
'deprecated',
'disjoint_base',
'Doc',
'evaluate_forward_ref',
'get_overloads',
'final',
'Format',
'get_annotations',
'get_args',
'get_origin',
'get_original_bases',
'get_protocol_members',
'get_type_hints',
'IntVar',
'is_protocol',
'is_typeddict',
'Literal',
'NewType',
'overload',
'override',
'Protocol',
'Sentinel',

```
'reveal_type',
'runtime',
'runtime_checkable',
'Text',
'TypeAlias',
'TypeAliasType',
'TypeForm',
'TypeGuard',
'TypeIs',
'TYPE_CHECKING',
'type_repr',
'Never',
'NoReturn',
'ReadOnly',
'Required',
'NotRequired',
'NoDefault',
'NoExtraItems',

# Pure aliases, have always been in typing
'AbstractSet',
'AnyStr',
'BinaryIO',
'Callable',
'Collection',
'Container',
'Dict',
'ForwardRef',
'FrozenSet',
'Generator',
'Generic',
'Hashable',
'IO',
'ItemsView',
'Iterable',
'Iterator',
'KeysView',
'List',
'Mapping',
'MappingView',
'Match',
'MutableMapping',
'MutableSequence',
'MutableSet',
```

```
    'Optional',
    'Pattern',
    'Reversible',
    'Sequence',
    'Set',
    'Sized',
    'TextIO',
    'Tuple',
    'Union',
    'ValuesView',
    'cast',
    'no_type_check',
    'no_type_check_decorator',
]

# for backward compatibility
PEP_560 = True
GenericMeta = type
# Breakpoint: https://github.com/python/cpython/pull/116129
_PEP_696_IMPLEMENTED = sys.version_info >= (3, 13, 0, "beta")

# Added with bpo-45166 to 3.10.1+ and some 3.9 versions
_FORWARD_REF_HAS_CLASS = "__forward_is_class__" in typing.ForwardRef.__slots__

# The functions below are modified copies of typing internal helpers.
# They are needed by _ProtocolMeta and they provide support for PEP 646.


class _Sentinel:
    def __repr__(self):
        return "<sentinel>"


_marker = _Sentinel()


# Breakpoint: https://github.com/python/cpython/pull/27342
if sys.version_info >= (3, 10):
    def _should_collect_from_parameters(t):
        return isinstance(
            t, (typing._GenericAlias, _types.GenericAlias, _types.UnionType)
        )
else:
    def _should_collect_from_parameters(t):
```

```
        return isinstance(t, (typing._GenericAlias, _types.GenericAlias))


NoReturn = typing.NoReturn

# Some unconstrained type variables.  These are used by the container types.
# (These are not for export.)
T = typing.TypeVar('T')  # Any type.
KT = typing.TypeVar('KT')  # Key type.
VT = typing.TypeVar('VT')  # Value type.
T_co = typing.TypeVar('T_co', covariant=True)  # Any type covariant containers.
T_contra = typing.TypeVar('T_contra', contravariant=True)  # Ditto contravariant.


# Breakpoint: https://github.com/python/cpython/pull/31841
if sys.version_info >= (3, 11):
    from typing import Any
else:

    class _AnyMeta(type):
        def __instancecheck__(self, obj):
            if self is Any:
                raise TypeError("typing_extensions.Any cannot be used with isinstance()")
            return super().__instancecheck__(obj)

        def __repr__(self):
            if self is Any:
                return "typing_extensions.Any"
            return super().__repr__()

    class Any(metaclass=_AnyMeta):
        """Special type indicating an unconstrained type.
        - Any is compatible with every type.
        - Any assumed to have all methods.
        - All values assumed to be instances of Any.
        Note that all the above statements are true from the point of view of
        static type checkers. At runtime, Any should not be used with instance
        checks.
        """
        def __new__(cls, *args, **kwargs):
            if cls is Any:
                raise TypeError("Any cannot be instantiated")
            return super().__new__(cls, *args, **kwargs)
```

```
ClassVar = typing.ClassVar

# Vendored from cpython typing._SpecialFrom
# Having a separate class means that instances will not be rejected by
# typing._type_check.
class _SpecialForm(typing._Final, _root=True):
    __slots__ = ('_name', '__doc__', '_getitem')

    def __init__(self, getitem):
        self._getitem = getitem
        self._name = getitem.__name__
        self.__doc__ = getitem.__doc__

    def __getattr__(self, item):
        if item in {'__name__', '__qualname__'}:
            return self._name

        raise AttributeError(item)

    def __mro_entries__(self, bases):
        raise TypeError(f"Cannot subclass {self!r}")

    def __repr__(self):
        return f'typing_extensions.{self._name}'

    def __reduce__(self):
        return self._name

    def __call__(self, *args, **kwds):
        raise TypeError(f"Cannot instantiate {self!r}")

    def __or__(self, other):
        return typing.Union[self, other]

    def __ror__(self, other):
        return typing.Union[other, self]

    def __instancecheck__(self, obj):
        raise TypeError(f"{self} cannot be used with isinstance()")

    def __subclasscheck__(self, cls):
        raise TypeError(f"{self} cannot be used with issubclass()")
```

```
    @typing._tp_cache
    def __getitem__(self, parameters):
        return self._getitem(self, parameters)


# Note that inheriting from this class means that the object will be
# rejected by typing._type_check, so do not use it if the special form
# is arguably valid as a type by itself.
class _ExtensionsSpecialForm(typing._SpecialForm, _root=True):
    def __repr__(self):
        return 'typing_extensions.' + self._name


Final = typing.Final

# Breakpoint: https://github.com/python/cpython/pull/30530
if sys.version_info >= (3, 11):
    final = typing.final
else:
    # @final exists in 3.8+, but we backport it for all versions
    # before 3.11 to keep support for the __final__ attribute.
    # See https://bugs.python.org/issue46342
    def final(f):
        """This decorator can be used to indicate to type checkers that
        the decorated method cannot be overridden, and decorated class
        cannot be subclassed. For example:

            class Base:
                @final
                def done(self) -> None:
                    ...
            class Sub(Base):
                def done(self) -> None:  # Error reported by type checker
                    ...
            @final
            class Leaf:
                ...
            class Other(Leaf):  # Error reported by type checker
                ...

        There is no runtime checking of these properties. The decorator
        sets the ``__final__`` attribute to ``True`` on the decorated object
        to allow runtime introspection.
        """
```

```
        try:
            f.__final__ = True
        except (AttributeError, TypeError):
            # Skip the attribute silently if it is not writable.
            # AttributeError happens if the object has __slots__ or a
            # read-only property, TypeError if it's a builtin class.
            pass
        return f


if hasattr(typing, "disjoint_base"):  # 3.15
    disjoint_base = typing.disjoint_base
else:
    def disjoint_base(cls):
        """This decorator marks a class as a disjoint base.

        Child classes of a disjoint base cannot inherit from other disjoint bases that are
        not parent classes of the disjoint base.

        For example:

            @disjoint_base
            class Disjoint1: pass

            @disjoint_base
            class Disjoint2: pass

            class Disjoint3(Disjoint1, Disjoint2): pass  # Type checker error

        Type checkers can use knowledge of disjoint bases to detect unreachable code
        and determine when two types can overlap.

        See PEP 800."""
        cls.__disjoint_base__ = True
        return cls


def IntVar(name):
    return typing.TypeVar(name)


# A Literal bug was fixed in 3.11.0, 3.10.1 and 3.9.8
# Breakpoint: https://github.com/python/cpython/pull/29334
if sys.version_info >= (3, 10, 1):
```

```
    Literal = typing.Literal
else:
    def _flatten_literal_params(parameters):
        """An internal helper for Literal creation: flatten Literals among parameters"""
        params = []
        for p in parameters:
            if isinstance(p, _LiteralGenericAlias):
                params.extend(p.__args__)
            else:
                params.append(p)
        return tuple(params)

    def _value_and_type_iter(params):
        for p in params:
            yield p, type(p)

    class _LiteralGenericAlias(typing._GenericAlias, _root=True):
        def __eq__(self, other):
            if not isinstance(other, _LiteralGenericAlias):
                return NotImplemented
            these_args_deduped = set(_value_and_type_iter(self.__args__))
            other_args_deduped = set(_value_and_type_iter(other.__args__))
            return these_args_deduped == other_args_deduped

        def __hash__(self):
            return hash(frozenset(_value_and_type_iter(self.__args__)))

    class _LiteralForm(_ExtensionsSpecialForm, _root=True):
        def __init__(self, doc: str):
            self._name = 'Literal'
            self._doc = self.__doc__ = doc

        def __getitem__(self, parameters):
            if not isinstance(parameters, tuple):
                parameters = (parameters,)

            parameters = _flatten_literal_params(parameters)

            val_type_pairs = list(_value_and_type_iter(parameters))
            try:
                deduped_pairs = set(val_type_pairs)
            except TypeError:
                # unhashable parameters
                pass
```

```
            else:
                # similar logic to typing._deduplicate on Python 3.9+
                if len(deduped_pairs) < len(val_type_pairs):
                    new_parameters = []
                    for pair in val_type_pairs:
                        if pair in deduped_pairs:
                            new_parameters.append(pair[0])
                            deduped_pairs.remove(pair)
                    assert not deduped_pairs, deduped_pairs
                    parameters = tuple(new_parameters)

            return _LiteralGenericAlias(self, parameters)

        Literal = _LiteralForm(doc="""\
                             A type that can be used to indicate to type checkers
                             that the corresponding value has a value literally equivalent
                             to the provided parameter. For example:

                                 var: Literal[4] = 4

                             The type checker understands that 'var' is literally equal to
                             the value 4 and no other value.

                             Literal[...] cannot be subclassed. There is no runtime
                             checking verifying that the parameter is actually a value
                             instead of a type.""")


_overload_dummy = typing._overload_dummy


if hasattr(typing, "get_overloads"):  # 3.11+
    overload = typing.overload
    get_overloads = typing.get_overloads
    clear_overloads = typing.clear_overloads
else:
    # {module: {qualname: {firstlineno: func}}}
    _overload_registry = collections.defaultdict(
        functools.partial(collections.defaultdict, dict)
    )

    def overload(func):
        """Decorator for overloaded functions/methods.
```

```
    In a stub file, place two or more stub definitions for the same
    function in a row, each decorated with @overload.  For example:

    @overload
    def utf8(value: None) -> None: ...
    @overload
    def utf8(value: bytes) -> bytes: ...
    @overload
    def utf8(value: str) -> bytes: ...

    In a non-stub file (i.e. a regular .py file), do the same but
    follow it with an implementation.  The implementation should *not*
    be decorated with @overload.  For example:

    @overload
    def utf8(value: None) -> None: ...
    @overload
    def utf8(value: bytes) -> bytes: ...
    @overload
    def utf8(value: str) -> bytes: ...
    def utf8(value):
        # implementation goes here

    The overloads for a function can be retrieved at runtime using the
    get_overloads() function.
    """
    # classmethod and staticmethod
    f = getattr(func, "__func__", func)
    try:
        _overload_registry[f.__module__][f.__qualname__][
            f.__code__.co_firstlineno
        ] = func
    except AttributeError:
        # Not a normal function; ignore.
        pass
    return _overload_dummy

def get_overloads(func):
    """Return all defined overloads for *func* as a sequence."""
    # classmethod and staticmethod
    f = getattr(func, "__func__", func)
    if f.__module__ not in _overload_registry:
        return []
    mod_dict = _overload_registry[f.__module__]
```

```
        if f.__qualname__ not in mod_dict:
            return []
        return list(mod_dict[f.__qualname__].values())

    def clear_overloads():
        """Clear all overloads in the registry."""
        _overload_registry.clear()


# This is not a real generic class.  Don't use outside annotations.
Type = typing.Type

# Various ABCs mimicking those in collections.abc.
# A few are simply re-exported for completeness.
Awaitable = typing.Awaitable
Coroutine = typing.Coroutine
AsyncIterable = typing.AsyncIterable
AsyncIterator = typing.AsyncIterator
Deque = typing.Deque
DefaultDict = typing.DefaultDict
OrderedDict = typing.OrderedDict
Counter = typing.Counter
ChainMap = typing.ChainMap
Text = typing.Text
TYPE_CHECKING = typing.TYPE_CHECKING


# Breakpoint: https://github.com/python/cpython/pull/118681
if sys.version_info >= (3, 13, 0, "beta"):
    from typing import AsyncContextManager, AsyncGenerator, ContextManager, Generator
else:
    def _is_dunder(attr):
        return attr.startswith('__') and attr.endswith('__')


    class _SpecialGenericAlias(typing._SpecialGenericAlias, _root=True):
        def __init__(self, origin, nparams, *, inst=True, name=None, defaults=()):
            super().__init__(origin, nparams, inst=inst, name=name)
            self._defaults = defaults

        def __setattr__(self, attr, val):
            allowed_attrs = {'_name', '_inst', '_nparams', '_defaults'}
            if _is_dunder(attr) or attr in allowed_attrs:
                object.__setattr__(self, attr, val)
```

```
        else:
            setattr(self.__origin__, attr, val)

    @typing._tp_cache
    def __getitem__(self, params):
        if not isinstance(params, tuple):
            params = (params,)
        msg = "Parameters to generic types must be types."
        params = tuple(typing._type_check(p, msg) for p in params)
        if (
            self._defaults
            and len(params) < self._nparams
            and len(params) + len(self._defaults) >= self._nparams
        ):
            params = (*params, *self._defaults[len(params) - self._nparams:])
        actual_len = len(params)

        if actual_len != self._nparams:
            if self._defaults:
                expected = f"at least {self._nparams - len(self._defaults)}"
            else:
                expected = str(self._nparams)
            if not self._nparams:
                raise TypeError(f"{self} is not a generic class")
            raise TypeError(
                f"Too {'many' if actual_len > self._nparams else 'few'}"
                f" arguments for {self};"
                f" actual {actual_len}, expected {expected}"
            )
        return self.copy_with(params)

_NoneType = type(None)
Generator = _SpecialGenericAlias(
    collections.abc.Generator, 3, defaults=(_NoneType, _NoneType)
)
AsyncGenerator = _SpecialGenericAlias(
    collections.abc.AsyncGenerator, 2, defaults=(_NoneType,)
)
ContextManager = _SpecialGenericAlias(
    contextlib.AbstractContextManager,
    2,
    name="ContextManager",
    defaults=(typing.Optional[bool],)
)
```

```
    AsyncContextManager = _SpecialGenericAlias(
        contextlib.AbstractAsyncContextManager,
        2,
        name="AsyncContextManager",
        defaults=(typing.Optional[bool],)
    )


_PROTO_ALLOWLIST = {
    'collections.abc': [
        'Callable', 'Awaitable', 'Iterable', 'Iterator', 'AsyncIterable',
        'Hashable', 'Sized', 'Container', 'Collection', 'Reversible', 'Buffer',
    ],
    'contextlib': ['AbstractContextManager', 'AbstractAsyncContextManager'],
    'typing_extensions': ['Buffer'],
}


_EXCLUDED_ATTRS = frozenset(typing.EXCLUDED_ATTRIBUTES) | {
    "__match_args__", "__protocol_attrs__", "__non_callable_proto_members__",
    "__final__",
}


def _get_protocol_attrs(cls):
    attrs = set()
    for base in cls.__mro__[:-1]:  # without object
        if base.__name__ in {'Protocol', 'Generic'}:
            continue
        annotations = getattr(base, '__annotations__', {})
        for attr in (*base.__dict__, *annotations):
            if (not attr.startswith('_abc_') and attr not in _EXCLUDED_ATTRS):
                attrs.add(attr)
    return attrs


def _caller(depth=1, default='__main__'):
    try:
        return sys._getframemodulename(depth + 1) or default
    except AttributeError:  # For platforms without _getframemodulename()
        pass
    try:
        return sys._getframe(depth + 1).f_globals.get('__name__', default)
    except (AttributeError, ValueError):  # For platforms without _getframe()
```

```
            pass
    return None


# `__match_args__` attribute was removed from protocol members in 3.13,
# we want to backport this change to older Python versions.
# Breakpoint: https://github.com/python/cpython/pull/110683
if sys.version_info >= (3, 13):
    Protocol = typing.Protocol
else:
    def _allow_reckless_class_checks(depth=2):
        """Allow instance and class checks for special stdlib modules.
        The abc and functools modules indiscriminately call isinstance() and
        issubclass() on the whole MRO of a user class, which may contain protocols.
        """
        return _caller(depth) in {'abc', 'functools', None}

    def _no_init(self, *args, **kwargs):
        if type(self)._is_protocol:
            raise TypeError('Protocols cannot be instantiated')

    def _type_check_issubclass_arg_1(arg):
        """Raise TypeError if `arg` is not an instance of `type`
        in `issubclass(arg, <protocol>)`.

        In most cases, this is verified by type.__subclasscheck__.
        Checking it again unnecessarily would slow down issubclass() checks,
        so, we don't perform this check unless we absolutely have to.

        For various error paths, however,
        we want to ensure that *this* error message is shown to the user
        where relevant, rather than a typing.py-specific error message.
        """
        if not isinstance(arg, type):
            # Same error message as for issubclass(1, int).
            raise TypeError('issubclass() arg 1 must be a class')

    # Inheriting from typing._ProtocolMeta isn't actually desirable,
    # but is necessary to allow typing.Protocol and typing_extensions.Protocol
    # to mix without getting TypeErrors about "metaclass conflict"
    class _ProtocolMeta(type(typing.Protocol)):
        # This metaclass is somewhat unfortunate,
        # but is necessary for several reasons...
        #
```

```
# NOTE: DO NOT call super() in any methods in this class
# That would call the methods on typing._ProtocolMeta on Python <=3.11
# and those are slow
def __new__(mcls, name, bases, namespace, **kwargs):
    if name == "Protocol" and len(bases) < 2:
        pass
    elif {Protocol, typing.Protocol} & set(bases):
        for base in bases:
            if not (
                base in {object, typing.Generic, Protocol, typing.Protocol}
                or base.__name__ in _PROTO_ALLOWLIST.get(base.__module__, [])
                or is_protocol(base)
            ):
                raise TypeError(
                    f"Protocols can only inherit from other protocols, "
                    f"got {base!r}"
                )
    return abc.ABCMeta.__new__(mcls, name, bases, namespace, **kwargs)

def __init__(cls, *args, **kwargs):
    abc.ABCMeta.__init__(cls, *args, **kwargs)
    if getattr(cls, "_is_protocol", False):
        cls.__protocol_attrs__ = _get_protocol_attrs(cls)

def __subclasscheck__(cls, other):
    if cls is Protocol:
        return type.__subclasscheck__(cls, other)
    if (
        getattr(cls, '_is_protocol', False)
        and not _allow_reckless_class_checks()
    ):
        if not getattr(cls, '_is_runtime_protocol', False):
            _type_check_issubclass_arg_1(other)
            raise TypeError(
                "Instance and class checks can only be used with "
                "@runtime_checkable protocols"
            )
        if (
            # this attribute is set by @runtime_checkable:
            cls.__non_callable_proto_members__
            and cls.__dict__.get("__subclasshook__") is _proto_hook
        ):
            _type_check_issubclass_arg_1(other)
            non_method_attrs = sorted(cls.__non_callable_proto_members__)
```

```
            raise TypeError(
                "Protocols with non-method members don't support issubclass()."
                f" Non-method members: {str(non_method_attrs)[1:-1]}."
            )
    return abc.ABCMeta.__subclasscheck__(cls, other)

def __instancecheck__(cls, instance):
    # We need this method for situations where attributes are
    # assigned in __init__.
    if cls is Protocol:
        return type.__instancecheck__(cls, instance)
    if not getattr(cls, "_is_protocol", False):
        # i.e., it's a concrete subclass of a protocol
        return abc.ABCMeta.__instancecheck__(cls, instance)

    if (
        not getattr(cls, '_is_runtime_protocol', False) and
        not _allow_reckless_class_checks()
    ):
        raise TypeError("Instance and class checks can only be used with"
                        " @runtime_checkable protocols")

    if abc.ABCMeta.__instancecheck__(cls, instance):
        return True

    for attr in cls.__protocol_attrs__:
        try:
            val = inspect.getattr_static(instance, attr)
        except AttributeError:
            break
        # this attribute is set by @runtime_checkable:
        if val is None and attr not in cls.__non_callable_proto_members__:
            break
    else:
        return True

    return False

def __eq__(cls, other):
    # Hack so that typing.Generic.__class_getitem__
    # treats typing_extensions.Protocol
    # as equivalent to typing.Protocol
    if abc.ABCMeta.__eq__(cls, other) is True:
        return True
```

```
        return cls is Protocol and other is typing.Protocol

    # This has to be defined, or the abc-module cache
    # complains about classes with this metaclass being unhashable,
    # if we define only __eq__!
    def __hash__(cls) -> int:
        return type.__hash__(cls)

@classmethod
def _proto_hook(cls, other):
    if not cls.__dict__.get('_is_protocol', False):
        return NotImplemented

    for attr in cls.__protocol_attrs__:
        for base in other.__mro__:
            # Check if the members appears in the class dictionary...
            if attr in base.__dict__:
                if base.__dict__[attr] is None:
                    return NotImplemented
                break

            # ...or in annotations, if it is a sub-protocol.
            annotations = getattr(base, '__annotations__', {})
            if (
                isinstance(annotations, collections.abc.Mapping)
                and attr in annotations
                and is_protocol(other)
            ):
                break
        else:
            return NotImplemented
    return True

class Protocol(typing.Generic, metaclass=_ProtocolMeta):
    __doc__ = typing.Protocol.__doc__
    __slots__ = ()
    _is_protocol = True
    _is_runtime_protocol = False

    def __init_subclass__(cls, *args, **kwargs):
        super().__init_subclass__(*args, **kwargs)

        # Determine if this is a protocol or a concrete subclass.
        if not cls.__dict__.get('_is_protocol', False):
```

```
            cls._is_protocol = any(b is Protocol for b in cls.__bases__)

        # Set (or override) the protocol subclass hook.
        if '__subclasshook__' not in cls.__dict__:
            cls.__subclasshook__ = _proto_hook

        # Prohibit instantiation for protocol classes
        if cls._is_protocol and cls.__init__ is Protocol.__init__:
            cls.__init__ = _no_init


# Breakpoint: https://github.com/python/cpython/pull/113401
if sys.version_info >= (3, 13):
    runtime_checkable = typing.runtime_checkable
else:
    def runtime_checkable(cls):
        """Mark a protocol class as a runtime protocol.

        Such protocol can be used with isinstance() and issubclass().
        Raise TypeError if applied to a non-protocol class.
        This allows a simple-minded structural check very similar to
        one trick ponies in collections.abc such as Iterable.

        For example::

            @runtime_checkable
            class Closable(Protocol):
                def close(self): ...

            assert isinstance(open('/some/file'), Closable)

        Warning: this will check only the presence of the required methods,
        not their type signatures!
        """
        if not issubclass(cls, typing.Generic) or not getattr(cls, '_is_protocol', False):
            raise TypeError(f'@runtime_checkable can be only applied to protocol classes,'
                            f' got {cls!r}')
        cls._is_runtime_protocol = True

        # typing.Protocol classes on <=3.11 break if we execute this block,
        # because typing.Protocol classes on <=3.11 don't have a
        # `__protocol_attrs__` attribute, and this block relies on the
        # `__protocol_attrs__` attribute. Meanwhile, typing.Protocol classes on 3.12.2+
        # break if we *don't* execute this block, because *they* assume that all
```

```
        # protocol classes have a `__non_callable_proto_members__` attribute
        # (which this block sets)
        if isinstance(cls, _ProtocolMeta) or sys.version_info >= (3, 12, 2):
            # PEP 544 prohibits using issubclass()
            # with protocols that have non-method members.
            # See gh-113320 for why we compute this attribute here,
            # rather than in `_ProtocolMeta.__init__`
            cls.__non_callable_proto_members__ = set()
            for attr in cls.__protocol_attrs__:
                try:
                    is_callable = callable(getattr(cls, attr, None))
                except Exception as e:
                    raise TypeError(
                        f"Failed to determine whether protocol member {attr!r} "
                        "is a method member"
                    ) from e
                else:
                    if not is_callable:
                        cls.__non_callable_proto_members__.add(attr)

        return cls


# The "runtime" alias exists for backwards compatibility.
runtime = runtime_checkable


# Our version of runtime-checkable protocols is faster on Python <=3.11
# Breakpoint: https://github.com/python/cpython/pull/112717
if sys.version_info >= (3, 12):
    SupportsInt = typing.SupportsInt
    SupportsFloat = typing.SupportsFloat
    SupportsComplex = typing.SupportsComplex
    SupportsBytes = typing.SupportsBytes
    SupportsIndex = typing.SupportsIndex
    SupportsAbs = typing.SupportsAbs
    SupportsRound = typing.SupportsRound
else:
    @runtime_checkable
    class SupportsInt(Protocol):
        """An ABC with one abstract method __int__."""
        __slots__ = ()

        @abc.abstractmethod
```

```
    def __int__(self) -> int:
        pass

@runtime_checkable
class SupportsFloat(Protocol):
    """An ABC with one abstract method __float__."""
    __slots__ = ()

    @abc.abstractmethod
    def __float__(self) -> float:
        pass

@runtime_checkable
class SupportsComplex(Protocol):
    """An ABC with one abstract method __complex__."""
    __slots__ = ()

    @abc.abstractmethod
    def __complex__(self) -> complex:
        pass

@runtime_checkable
class SupportsBytes(Protocol):
    """An ABC with one abstract method __bytes__."""
    __slots__ = ()

    @abc.abstractmethod
    def __bytes__(self) -> bytes:
        pass

@runtime_checkable
class SupportsIndex(Protocol):
    __slots__ = ()

    @abc.abstractmethod
    def __index__(self) -> int:
        pass

@runtime_checkable
class SupportsAbs(Protocol[T_co]):
    """
    An ABC with one abstract method __abs__ that is covariant in its return type.
    """
    __slots__ = ()
```

```
        @abc.abstractmethod
        def __abs__(self) -> T_co:
            pass

    @runtime_checkable
    class SupportsRound(Protocol[T_co]):
        """
        An ABC with one abstract method __round__ that is covariant in its return type.
        """
        __slots__ = ()

        @abc.abstractmethod
        def __round__(self, ndigits: int = 0) -> T_co:
            pass


if hasattr(io, "Reader") and hasattr(io, "Writer"):
    Reader = io.Reader
    Writer = io.Writer
else:
    @runtime_checkable
    class Reader(Protocol[T_co]):
        """Protocol for simple I/O reader instances.

        This protocol only supports blocking I/O.
        """

        __slots__ = ()

        @abc.abstractmethod
        def read(self, size: int = ..., /) -> T_co:
            """Read data from the input stream and return it.

            If *size* is specified, at most *size* items (bytes/characters) will be
            read.
            """

    @runtime_checkable
    class Writer(Protocol[T_contra]):
        """Protocol for simple I/O writer instances.

        This protocol only supports blocking I/O.
        """
```

```
        __slots__ = ()

        @abc.abstractmethod
        def write(self, data: T_contra, /) -> int:
            """Write *data* to the output stream and return the number of items written.""" #
noqa: E501


_NEEDS_SINGLETONMETA = (
    not hasattr(typing, "NoDefault") or not hasattr(typing, "NoExtraItems")
)

if _NEEDS_SINGLETONMETA:
    class SingletonMeta(type):
        def __setattr__(cls, attr, value):
            # TypeError is consistent with the behavior of NoneType
            raise TypeError(
                f"cannot set {attr!r} attribute of immutable type {cls.__name__!r}"
            )


if hasattr(typing, "NoDefault"):
    NoDefault = typing.NoDefault
else:
    class NoDefaultType(metaclass=SingletonMeta):
        """The type of the NoDefault singleton."""

        __slots__ = ()

        def __new__(cls):
            return globals().get("NoDefault") or object.__new__(cls)

        def __repr__(self):
            return "typing_extensions.NoDefault"

        def __reduce__(self):
            return "NoDefault"

    NoDefault = NoDefaultType()
    del NoDefaultType

if hasattr(typing, "NoExtraItems"):
    NoExtraItems = typing.NoExtraItems
```

```
else:
    class NoExtraItemsType(metaclass=SingletonMeta):
        """The type of the NoExtraItems singleton."""

        __slots__ = ()

        def __new__(cls):
            return globals().get("NoExtraItems") or object.__new__(cls)

        def __repr__(self):
            return "typing_extensions.NoExtraItems"

        def __reduce__(self):
            return "NoExtraItems"

    NoExtraItems = NoExtraItemsType()
    del NoExtraItemsType

if _NEEDS_SINGLETONMETA:
    del SingletonMeta


# Update this to something like >=3.13.0b1 if and when
# PEP 728 is implemented in CPython
_PEP_728_IMPLEMENTED = False

if _PEP_728_IMPLEMENTED:
    # The standard library TypedDict in Python 3.9.0/1 does not honour the "total"
    # keyword with old-style TypedDict().  See https://bugs.python.org/issue42059
    # The standard library TypedDict below Python 3.11 does not store runtime
    # information about optional and required keys when using Required or NotRequired.
    # Generic TypedDicts are also impossible using typing.TypedDict on Python <3.11.
    # Aaaand on 3.12 we add __orig_bases__ to TypedDict
    # to enable better runtime introspection.
    # On 3.13 we deprecate some odd ways of creating TypedDicts.
    # Also on 3.13, PEP 705 adds the ReadOnly[] qualifier.
    # PEP 728 (still pending) makes more changes.
    TypedDict = typing.TypedDict
    _TypedDictMeta = typing._TypedDictMeta
    is_typeddict = typing.is_typeddict
else:
    # 3.10.0 and later
    _TAKES_MODULE = "module" in inspect.signature(typing._type_check).parameters
```

```
def _get_typeddict_qualifiers(annotation_type):
    while True:
        annotation_origin = get_origin(annotation_type)
        if annotation_origin is Annotated:
            annotation_args = get_args(annotation_type)
            if annotation_args:
                annotation_type = annotation_args[0]
            else:
                break
        elif annotation_origin is Required:
            yield Required
            annotation_type, = get_args(annotation_type)
        elif annotation_origin is NotRequired:
            yield NotRequired
            annotation_type, = get_args(annotation_type)
        elif annotation_origin is ReadOnly:
            yield ReadOnly
            annotation_type, = get_args(annotation_type)
        else:
            break

class _TypedDictMeta(type):

    def __new__(cls, name, bases, ns, *, total=True, closed=None,
                extra_items=NoExtraItems):
        """Create new typed dict class object.

        This method is called when TypedDict is subclassed,
        or when TypedDict is instantiated. This way
        TypedDict supports all three syntax forms described in its docstring.
        Subclasses and instances of TypedDict return actual dictionaries.
        """
        for base in bases:
            if type(base) is not _TypedDictMeta and base is not typing.Generic:
                raise TypeError('cannot inherit from both a TypedDict type '
                                'and a non-TypedDict base class')
        if closed is not None and extra_items is not NoExtraItems:
            raise TypeError(f"Cannot combine closed={closed!r} and extra_items")

        if any(issubclass(b, typing.Generic) for b in bases):
            generic_base = (typing.Generic,)
        else:
            generic_base = ()
```

```
ns_annotations = ns.pop('__annotations__', None)

# typing.py generally doesn't let you inherit from plain Generic, unless
# the name of the class happens to be "Protocol"
tp_dict = type.__new__(_TypedDictMeta, "Protocol", (*generic_base, dict), ns)
tp_dict.__name__ = name
if tp_dict.__qualname__ == "Protocol":
    tp_dict.__qualname__ = name

if not hasattr(tp_dict, '__orig_bases__'):
    tp_dict.__orig_bases__ = bases

annotations = {}
own_annotate = None
if ns_annotations is not None:
    own_annotations = ns_annotations
elif sys.version_info >= (3, 14):
    if hasattr(annotationlib, "get_annotate_from_class_namespace"):
        own_annotate = annotationlib.get_annotate_from_class_namespace(ns)
    else:
        # 3.14.0a7 and earlier
        own_annotate = ns.get("__annotate__")
    if own_annotate is not None:
        own_annotations = annotationlib.call_annotate_function(
            own_annotate, Format.FORWARDREF, owner=tp_dict
        )
    else:
        own_annotations = {}
else:
    own_annotations = {}
msg = "TypedDict('Name', {f0: t0, f1: t1, ...}); each t must be a type"
if _TAKES_MODULE:
    own_checked_annotations = {
        n: typing._type_check(tp, msg, module=tp_dict.__module__)
        for n, tp in own_annotations.items()
    }
else:
    own_checked_annotations = {
        n: typing._type_check(tp, msg)
        for n, tp in own_annotations.items()
    }
required_keys = set()
optional_keys = set()
readonly_keys = set()
```

```
mutable_keys = set()
extra_items_type = extra_items

for base in bases:
    base_dict = base.__dict__

    if sys.version_info <= (3, 14):
        annotations.update(base_dict.get('__annotations__', {}))
    required_keys.update(base_dict.get('__required_keys__', ()))
    optional_keys.update(base_dict.get('__optional_keys__', ()))
    readonly_keys.update(base_dict.get('__readonly_keys__', ()))
    mutable_keys.update(base_dict.get('__mutable_keys__', ()))

# This was specified in an earlier version of PEP 728. Support
# is retained for backwards compatibility, but only for Python
# 3.13 and lower.
if (closed and sys.version_info < (3, 14)
          and "__extra_items__" in own_checked_annotations):
    annotation_type = own_checked_annotations.pop("__extra_items__")
    qualifiers = set(_get_typeddict_qualifiers(annotation_type))
    if Required in qualifiers:
        raise TypeError(
            "Special key __extra_items__ does not support "
            "Required"
        )
    if NotRequired in qualifiers:
        raise TypeError(
            "Special key __extra_items__ does not support "
            "NotRequired"
        )
    extra_items_type = annotation_type

annotations.update(own_checked_annotations)
for annotation_key, annotation_type in own_checked_annotations.items():
    qualifiers = set(_get_typeddict_qualifiers(annotation_type))

    if Required in qualifiers:
        required_keys.add(annotation_key)
    elif NotRequired in qualifiers:
        optional_keys.add(annotation_key)
    elif total:
        required_keys.add(annotation_key)
    else:
        optional_keys.add(annotation_key)
```

```
    if ReadOnly in qualifiers:
        mutable_keys.discard(annotation_key)
        readonly_keys.add(annotation_key)
    else:
        mutable_keys.add(annotation_key)
        readonly_keys.discard(annotation_key)

# Breakpoint: https://github.com/python/cpython/pull/119891
if sys.version_info >= (3, 14):
    def __annotate__(format):
        annos = {}
        for base in bases:
            if base is Generic:
                continue
            base_annotate = base.__annotate__
            if base_annotate is None:
                continue
            base_annos = annotationlib.call_annotate_function(
                base_annotate, format, owner=base)
            annos.update(base_annos)
        if own_annotate is not None:
            own = annotationlib.call_annotate_function(
                own_annotate, format, owner=tp_dict)
            if format != Format.STRING:
                own = {
                    n: typing._type_check(tp, msg, module=tp_dict.__module__)
                    for n, tp in own.items()
                }
        elif format == Format.STRING:
            own = annotationlib.annotations_to_string(own_annotations)
        elif format in (Format.FORWARDREF, Format.VALUE):
            own = own_checked_annotations
        else:
            raise NotImplementedError(format)
        annos.update(own)
        return annos

    tp_dict.__annotate__ = __annotate__
else:
    tp_dict.__annotations__ = annotations
tp_dict.__required_keys__ = frozenset(required_keys)
tp_dict.__optional_keys__ = frozenset(optional_keys)
tp_dict.__readonly_keys__ = frozenset(readonly_keys)
tp_dict.__mutable_keys__ = frozenset(mutable_keys)
```

```
        tp_dict.__total__ = total
        tp_dict.__closed__ = closed
        tp_dict.__extra_items__ = extra_items_type
        return tp_dict

    __call__ = dict  # static method

    def __subclasscheck__(cls, other):
        # Typed dicts are only for static structural subtyping.
        raise TypeError('TypedDict does not support instance and class checks')

    __instancecheck__ = __subclasscheck__

_TypedDict = type.__new__(_TypedDictMeta, 'TypedDict', (), {})

def _create_typeddict(
    typename,
    fields,
    /,
    *,
    typing_is_inline,
    total,
    closed,
    extra_items,
    **kwargs,
):
    if fields is _marker or fields is None:
        if fields is _marker:
            deprecated_thing = (
                "Failing to pass a value for the 'fields' parameter"
            )
        else:
            deprecated_thing = "Passing `None` as the 'fields' parameter"

        example = f"`{typename} = TypedDict({typename!r}, {{}})`"
        deprecation_msg = (
            f"{deprecated_thing} is deprecated and will be disallowed in "
            "Python 3.15. To create a TypedDict class with 0 fields "
            "using the functional syntax, pass an empty dictionary, e.g. "
        ) + example + "."
        warnings.warn(deprecation_msg, DeprecationWarning, stacklevel=2)
        # Support a field called "closed"
        if closed is not False and closed is not True and closed is not None:
            kwargs["closed"] = closed
```

```
            closed = None
        # Or "extra_items"
        if extra_items is not NoExtraItems:
            kwargs["extra_items"] = extra_items
            extra_items = NoExtraItems
        fields = kwargs
    elif kwargs:
        raise TypeError("TypedDict takes either a dict or keyword arguments,"
                        " but not both")
    if kwargs:
        # Breakpoint: https://github.com/python/cpython/pull/104891
        if sys.version_info >= (3, 13):
            raise TypeError("TypedDict takes no keyword arguments")
        warnings.warn(
            "The kwargs-based syntax for TypedDict definitions is deprecated "
            "in Python 3.11, will be removed in Python 3.13, and may not be "
            "understood by third-party type checkers.",
            DeprecationWarning,
            stacklevel=2,
        )

    ns = {'__annotations__': dict(fields)}
    module = _caller(depth=4 if typing_is_inline else 2)
    if module is not None:
        # Setting correct module is necessary to make typed dict classes
        # pickleable.
        ns['__module__'] = module

    td = _TypedDictMeta(typename, (), ns, total=total, closed=closed,
                        extra_items=extra_items)
    td.__orig_bases__ = (TypedDict,)
    return td

class _TypedDictSpecialForm(_SpecialForm, _root=True):
    def __call__(
        self,
        typename,
        fields=_marker,
        /,
        *,
        total=True,
        closed=None,
        extra_items=NoExtraItems,
        **kwargs
```

```
    ):
        return _create_typeddict(
            typename,
            fields,
            typing_is_inline=False,
            total=total,
            closed=closed,
            extra_items=extra_items,
            **kwargs,
        )

    def __mro_entries__(self, bases):
        return (_TypedDict,)

@_TypedDictSpecialForm
def TypedDict(self, args):
    """A simple typed namespace. At runtime it is equivalent to a plain dict.

    TypedDict creates a dictionary type such that a type checker will expect all
    instances to have a certain set of keys, where each key is
    associated with a value of a consistent type. This expectation
    is not checked at runtime.

    Usage::

        class Point2D(TypedDict):
            x: int
            y: int
            label: str

        a: Point2D = {'x': 1, 'y': 2, 'label': 'good'}  # OK
        b: Point2D = {'z': 3, 'label': 'bad'}           # Fails type check

        assert Point2D(x=1, y=2, label='first') == dict(x=1, y=2, label='first')

    The type info can be accessed via the Point2D.__annotations__ dict, and
    the Point2D.__required_keys__ and Point2D.__optional_keys__ frozensets.
    TypedDict supports an additional equivalent form::

        Point2D = TypedDict('Point2D', {'x': int, 'y': int, 'label': str})

    By default, all keys must be present in a TypedDict. It is possible
    to override this by specifying totality::
```

```
        class Point2D(TypedDict, total=False):
            x: int
            y: int

        This means that a Point2D TypedDict can have any of the keys omitted. A type
        checker is only expected to support a literal False or True as the value of
        the total argument. True is the default, and makes all items defined in the
        class body be required.

        The Required and NotRequired special forms can also be used to mark
        individual keys as being required or not required::

            class Point2D(TypedDict):
                x: int  # the "x" key must always be present (Required is the default)
                y: NotRequired[int]  # the "y" key can be omitted

        See PEP 655 for more details on Required and NotRequired.
        """
        # This runs when creating inline TypedDicts:
        if not isinstance(args, dict):
            raise TypeError(
                "TypedDict[...] should be used with a single dict argument"
            )

        return _create_typeddict(
            "<inline TypedDict>",
            args,
            typing_is_inline=True,
            total=True,
            closed=True,
            extra_items=NoExtraItems,
        )

    _TYPEDDICT_TYPES = (typing._TypedDictMeta, _TypedDictMeta)

    def is_typeddict(tp):
        """Check if an annotation is a TypedDict class

        For example::
            class Film(TypedDict):
                title: str
                year: int

            is_typeddict(Film)  # => True
```

```
            is_typeddict(Union[list, str])  # => False
        """
        return isinstance(tp, _TYPEDDICT_TYPES)


if hasattr(typing, "assert_type"):
    assert_type = typing.assert_type

else:
    def assert_type(val, typ, /):
        """Assert (to the type checker) that the value is of the given type.

        When the type checker encounters a call to assert_type(), it
        emits an error if the value is not of the specified type::

            def greet(name: str) -> None:
                assert_type(name, str)  # ok
                assert_type(name, int)  # type checker error

        At runtime this returns the first argument unchanged and otherwise
        does nothing.
        """
        return val


if hasattr(typing, "ReadOnly"):  # 3.13+
    get_type_hints = typing.get_type_hints
else:  # <=3.13
    # replaces _strip_annotations()
    def _strip_extras(t):
        """Strips Annotated, Required and NotRequired from a given type."""
        if isinstance(t, typing._AnnotatedAlias):
            return _strip_extras(t.__origin__)
        if hasattr(t, "__origin__") and t.__origin__ in (Required, NotRequired, ReadOnly):
            return _strip_extras(t.__args__[0])
        if isinstance(t, typing._GenericAlias):
            stripped_args = tuple(_strip_extras(a) for a in t.__args__)
            if stripped_args == t.__args__:
                return t
            return t.copy_with(stripped_args)
        if hasattr(_types, "GenericAlias") and isinstance(t, _types.GenericAlias):
            stripped_args = tuple(_strip_extras(a) for a in t.__args__)
            if stripped_args == t.__args__:
                return t
```

```
            return _types.GenericAlias(t.__origin__, stripped_args)
        if hasattr(_types, "UnionType") and isinstance(t, _types.UnionType):
            stripped_args = tuple(_strip_extras(a) for a in t.__args__)
            if stripped_args == t.__args__:
                return t
            return functools.reduce(operator.or_, stripped_args)

        return t

    def get_type_hints(obj, globalns=None, localns=None, include_extras=False):
        """Return type hints for an object.

        This is often the same as obj.__annotations__, but it handles
        forward references encoded as string literals, adds Optional[t] if a
        default value equal to None is set and recursively replaces all
        'Annotated[T, ...]', 'Required[T]' or 'NotRequired[T]' with 'T'
        (unless 'include_extras=True').

        The argument may be a module, class, method, or function. The annotations
        are returned as a dictionary. For classes, annotations include also
        inherited members.

        TypeError is raised if the argument is not of a type that can contain
        annotations, and an empty dictionary is returned if no annotations are
        present.

        BEWARE -- the behavior of globalns and localns is counterintuitive
        (unless you are familiar with how eval() and exec() work).  The
        search order is locals first, then globals.

        - If no dict arguments are passed, an attempt is made to use the
          globals from obj (or the respective module's globals for classes),
          and these are also used as the locals.  If the object does not appear
          to have globals, an empty dictionary is used.

        - If one dict argument is passed, it is used for both globals and
          locals.

        - If two dict arguments are passed, they specify globals and
          locals, respectively.
        """
        hint = typing.get_type_hints(
            obj, globalns=globalns, localns=localns, include_extras=True
        )
```

```
    # Breakpoint: https://github.com/python/cpython/pull/30304
    if sys.version_info < (3, 11):
        _clean_optional(obj, hint, globalns, localns)
    if include_extras:
        return hint
    return {k: _strip_extras(t) for k, t in hint.items()}

_NoneType = type(None)

def _could_be_inserted_optional(t):
    """detects Union[..., None] pattern"""
    if not isinstance(t, typing._UnionGenericAlias):
        return False
    # Assume if last argument is not None they are user defined
    if t.__args__[-1] is not _NoneType:
        return False
    return True

# < 3.11
def _clean_optional(obj, hints, globalns=None, localns=None):
    # reverts injected Union[..., None] cases from typing.get_type_hints
    # when a None default value is used.
    # see https://github.com/python/typing_extensions/issues/310
    if not hints or isinstance(obj, type):
        return
    defaults = typing._get_defaults(obj)  # avoid accessing __annotations___
    if not defaults:
        return
    original_hints = obj.__annotations__
    for name, value in hints.items():
        # Not a Union[..., None] or replacement conditions not fullfilled
        if (not _could_be_inserted_optional(value)
            or name not in defaults
            or defaults[name] is not None
        ):
            continue
        original_value = original_hints[name]
        # value=NoneType should have caused a skip above but check for safety
        if original_value is None:
            original_value = _NoneType
        # Forward reference
        if isinstance(original_value, str):
            if globalns is None:
                if isinstance(obj, _types.ModuleType):
```

```
                globalns = obj.__dict__
            else:
                nsobj = obj
                # Find globalns for the unwrapped object.
                while hasattr(nsobj, '__wrapped__'):
                    nsobj = nsobj.__wrapped__
                globalns = getattr(nsobj, '__globals__', {})
            if localns is None:
                localns = globalns
        elif localns is None:
            localns = globalns

        original_value = ForwardRef(
            original_value,
            is_argument=not isinstance(obj, _types.ModuleType)
        )
        original_evaluated = typing._eval_type(original_value, globalns, localns)
        # Compare if values differ. Note that even if equal
        # value might be cached by typing._tp_cache contrary to original_evaluated
        if original_evaluated != value or (
            # 3.10: ForwardRefs of UnionType might be turned into _UnionGenericAlias
            hasattr(_types, "UnionType")
            and isinstance(original_evaluated, _types.UnionType)
            and not isinstance(value, _types.UnionType)
        ):
            hints[name] = original_evaluated

# Python 3.9 has get_origin() and get_args() but those implementations don't support
# ParamSpecArgs and ParamSpecKwargs, so only Python 3.10's versions will do.
# Breakpoint: https://github.com/python/cpython/pull/25298
if sys.version_info >= (3, 10):
    get_origin = typing.get_origin
    get_args = typing.get_args
# 3.9
else:
    def get_origin(tp):
        """Get the unsubscripted version of a type.

        This supports generic types, Callable, Tuple, Union, Literal, Final, ClassVar
        and Annotated. Return None for unsupported types. Examples::

            get_origin(Literal[42]) is Literal
            get_origin(int) is None
            get_origin(ClassVar[int]) is ClassVar
```

```
            get_origin(Generic) is Generic
            get_origin(Generic[T]) is Generic
            get_origin(Union[T, int]) is Union
            get_origin(List[Tuple[T, T]][int]) == list
            get_origin(P.args) is P
        """
        if isinstance(tp, typing._AnnotatedAlias):
            return Annotated
        if isinstance(tp, (typing._BaseGenericAlias, _types.GenericAlias,
                           ParamSpecArgs, ParamSpecKwargs)):
            return tp.__origin__
        if tp is typing.Generic:
            return typing.Generic
        return None

    def get_args(tp):
        """Get type arguments with all substitutions performed.

        For unions, basic simplifications used by Union constructor are performed.
        Examples::
            get_args(Dict[str, int]) == (str, int)
            get_args(int) == ()
            get_args(Union[int, Union[T, int], str][int]) == (int, str)
            get_args(Union[int, Tuple[T, int]][str]) == (int, Tuple[str, int])
            get_args(Callable[[], T][int]) == ([], int)
        """
        if isinstance(tp, typing._AnnotatedAlias):
            return (tp.__origin__, *tp.__metadata__)
        if isinstance(tp, (typing._GenericAlias, _types.GenericAlias)):
            res = tp.__args__
            if get_origin(tp) is collections.abc.Callable and res[0] is not Ellipsis:
                res = (list(res[:-1]), res[-1])
            return res
        return ()


# 3.10+
if hasattr(typing, 'TypeAlias'):
    TypeAlias = typing.TypeAlias
# 3.9
else:
    @_ExtensionsSpecialForm
    def TypeAlias(self, parameters):
        """Special marker indicating that an assignment should
```

```
        be recognized as a proper type alias definition by type
        checkers.

        For example::

            Predicate: TypeAlias = Callable[..., bool]

        It's invalid when used anywhere except as in the example above.
        """
        raise TypeError(f"{self} is not subscriptable")


def _set_default(type_param, default):
    type_param.has_default = lambda: default is not NoDefault
    type_param.__default__ = default


def _set_module(typevarlike):
    # for pickling:
    def_mod = _caller(depth=2)
    if def_mod != 'typing_extensions':
        typevarlike.__module__ = def_mod


class _DefaultMixin:
    """Mixin for TypeVarLike defaults."""

    __slots__ = ()
    __init__ = _set_default


# Classes using this metaclass must provide a _backported_typevarlike ClassVar
class _TypeVarLikeMeta(type):
    def __instancecheck__(cls, __instance: Any) -> bool:
        return isinstance(__instance, cls._backported_typevarlike)


if _PEP_696_IMPLEMENTED:
    from typing import TypeVar
else:
    # Add default and infer_variance parameters from PEP 696 and 695
    class TypeVar(metaclass=_TypeVarLikeMeta):
        """Type variable."""
```

```
        _backported_typevarlike = typing.TypeVar

        def __new__(cls, name, *constraints, bound=None,
                    covariant=False, contravariant=False,
                    default=NoDefault, infer_variance=False):
            if hasattr(typing, "TypeAliasType"):
                # PEP 695 implemented (3.12+), can pass infer_variance to typing.TypeVar
                typevar = typing.TypeVar(name, *constraints, bound=bound,
                                         covariant=covariant, contravariant=contravariant,
                                         infer_variance=infer_variance)
            else:
                typevar = typing.TypeVar(name, *constraints, bound=bound,
                                         covariant=covariant, contravariant=contravariant)
                if infer_variance and (covariant or contravariant):
                    raise ValueError("Variance cannot be specified with infer_variance.")
                typevar.__infer_variance__ = infer_variance

            _set_default(typevar, default)
            _set_module(typevar)

            def _tvar_prepare_subst(alias, args):
                if (
                    typevar.has_default()
                    and alias.__parameters__.index(typevar) == len(args)
                ):
                    args += (typevar.__default__,)
                return args

            typevar.__typing_prepare_subst__ = _tvar_prepare_subst
            return typevar

        def __init_subclass__(cls) -> None:
            raise TypeError(f"type '{__name__}.TypeVar' is not an acceptable base type")


# Python 3.10+ has PEP 612
if hasattr(typing, 'ParamSpecArgs'):
    ParamSpecArgs = typing.ParamSpecArgs
    ParamSpecKwargs = typing.ParamSpecKwargs
# 3.9
else:
    class _Immutable:
        """Mixin to indicate that object should not be copied."""
        __slots__ = ()
```

```
    def __copy__(self):
        return self

    def __deepcopy__(self, memo):
        return self

class ParamSpecArgs(_Immutable):
    """The args for a ParamSpec object.

    Given a ParamSpec object P, P.args is an instance of ParamSpecArgs.

    ParamSpecArgs objects have a reference back to their ParamSpec:

    P.args.__origin__ is P

    This type is meant for runtime introspection and has no special meaning to
    static type checkers.
    """
    def __init__(self, origin):
        self.__origin__ = origin

    def __repr__(self):
        return f"{self.__origin__.__name__}.args"

    def __eq__(self, other):
        if not isinstance(other, ParamSpecArgs):
            return NotImplemented
        return self.__origin__ == other.__origin__

class ParamSpecKwargs(_Immutable):
    """The kwargs for a ParamSpec object.

    Given a ParamSpec object P, P.kwargs is an instance of ParamSpecKwargs.

    ParamSpecKwargs objects have a reference back to their ParamSpec:

    P.kwargs.__origin__ is P

    This type is meant for runtime introspection and has no special meaning to
    static type checkers.
    """
    def __init__(self, origin):
        self.__origin__ = origin
```

```
        def __repr__(self):
            return f"{self.__origin__.__name__}.kwargs"

        def __eq__(self, other):
            if not isinstance(other, ParamSpecKwargs):
                return NotImplemented
            return self.__origin__ == other.__origin__


if _PEP_696_IMPLEMENTED:
    from typing import ParamSpec

# 3.10+
elif hasattr(typing, 'ParamSpec'):

    # Add default parameter - PEP 696
    class ParamSpec(metaclass=_TypeVarLikeMeta):
        """Parameter specification."""

        _backported_typevarlike = typing.ParamSpec

        def __new__(cls, name, *, bound=None,
                    covariant=False, contravariant=False,
                    infer_variance=False, default=NoDefault):
            if hasattr(typing, "TypeAliasType"):
                # PEP 695 implemented, can pass infer_variance to typing.TypeVar
                paramspec = typing.ParamSpec(name, bound=bound,
                                             covariant=covariant,
                                             contravariant=contravariant,
                                             infer_variance=infer_variance)
            else:
                paramspec = typing.ParamSpec(name, bound=bound,
                                             covariant=covariant,
                                             contravariant=contravariant)
                paramspec.__infer_variance__ = infer_variance

            _set_default(paramspec, default)
            _set_module(paramspec)

            def _paramspec_prepare_subst(alias, args):
                params = alias.__parameters__
                i = params.index(paramspec)
                if i == len(args) and paramspec.has_default():
```

```
                args = [*args, paramspec.__default__]
            if i >= len(args):
                raise TypeError(f"Too few arguments for {alias}")
            # Special case where Z[[int, str, bool]] == Z[int, str, bool] in PEP 612.
            if len(params) == 1 and not typing._is_param_expr(args[0]):
                assert i == 0
                args = (args,)
            # Convert lists to tuples to help other libraries cache the results.
            elif isinstance(args[i], list):
                args = (*args[:i], tuple(args[i]), *args[i + 1:])
            return args

        paramspec.__typing_prepare_subst__ = _paramspec_prepare_subst
        return paramspec

    def __init_subclass__(cls) -> None:
        raise TypeError(f"type '{__name__}.ParamSpec' is not an acceptable base type")

# 3.9
else:

    # Inherits from list as a workaround for Callable checks in Python < 3.9.2.
    class ParamSpec(list, _DefaultMixin):
        """Parameter specification variable.

        Usage::

          P = ParamSpec('P')

        Parameter specification variables exist primarily for the benefit of static
        type checkers.  They are used to forward the parameter types of one
        callable to another callable, a pattern commonly found in higher order
        functions and decorators.  They are only valid when used in ``Concatenate``,
        or s the first argument to ``Callable``. In Python 3.10 and higher,
        they are also supported in user-defined Generics at runtime.
        See class Generic for more information on generic types.  An
        example for annotating a decorator::

          T = TypeVar('T')
          P = ParamSpec('P')

          def add_logging(f: Callable[P, T]) -> Callable[P, T]:
              '''A type-safe decorator to add logging to a function.'''
              def inner(*args: P.args, **kwargs: P.kwargs) -> T:
```

```
        logging.info(f'{f.__name__} was called')
        return f(*args, **kwargs)
    return inner

@add_logging
def add_two(x: float, y: float) -> float:
    '''Add two numbers together.'''
    return x + y
```

Parameter specification variables defined with covariant=True or contravariant=True can be used to declare covariant or contravariant generic types. These keyword arguments are valid, but their actual semantics are yet to be decided. See PEP 612 for details.

Parameter specification variables can be introspected. e.g.:

```
P.__name__ == 'T'
P.__bound__ == None
P.__covariant__ == False
P.__contravariant__ == False
```

Note that only parameter specification variables defined in global scope can be pickled.
"""

```
# Trick Generic __parameters__.
__class__ = typing.TypeVar

@property
def args(self):
    return ParamSpecArgs(self)

@property
def kwargs(self):
    return ParamSpecKwargs(self)

def __init__(self, name, *, bound=None, covariant=False, contravariant=False,
             infer_variance=False, default=NoDefault):
    list.__init__(self, [self])
    self.__name__ = name
    self.__covariant__ = bool(covariant)
    self.__contravariant__ = bool(contravariant)
    self.__infer_variance__ = bool(infer_variance)
    if bound:
```

```
            self.__bound__ = typing._type_check(bound, 'Bound must be a type.')
        else:
            self.__bound__ = None
        _DefaultMixin.__init__(self, default)

        # for pickling:
        def_mod = _caller()
        if def_mod != 'typing_extensions':
            self.__module__ = def_mod

    def __repr__(self):
        if self.__infer_variance__:
            prefix = ''
        elif self.__covariant__:
            prefix = '+'
        elif self.__contravariant__:
            prefix = '-'
        else:
            prefix = '~'
        return prefix + self.__name__

    def __hash__(self):
        return object.__hash__(self)

    def __eq__(self, other):
        return self is other

    def __reduce__(self):
        return self.__name__

    # Hack to get typing._type_check to pass.
    def __call__(self, *args, **kwargs):
        pass


# 3.9
if not hasattr(typing, 'Concatenate'):
    # Inherits from list as a workaround for Callable checks in Python < 3.9.2.

    # 3.9.0-1
    if not hasattr(typing, '_type_convert'):
        def _type_convert(arg, module=None, *, allow_special_forms=False):
            """For converting None to type(None), and strings to ForwardRef."""
            if arg is None:
```

```
            return type(None)
        if isinstance(arg, str):
            if sys.version_info <= (3, 9, 6):
                return ForwardRef(arg)
            if sys.version_info <= (3, 9, 7):
                return ForwardRef(arg, module=module)
            return ForwardRef(arg, module=module, is_class=allow_special_forms)
        return arg
else:
    _type_convert = typing._type_convert

class _ConcatenateGenericAlias(list):

    # Trick Generic into looking into this for __parameters__.
    __class__ = typing._GenericAlias

    def __init__(self, origin, args):
        super().__init__(args)
        self.__origin__ = origin
        self.__args__ = args

    def __repr__(self):
        _type_repr = typing._type_repr
        return (f'{_type_repr(self.__origin__)}'
                f'[{", ".join(_type_repr(arg) for arg in self.__args__)}]')

    def __hash__(self):
        return hash((self.__origin__, self.__args__))

    # Hack to get typing._type_check to pass in Generic.
    def __call__(self, *args, **kwargs):
        pass

    @property
    def __parameters__(self):
        return tuple(
            tp for tp in self.__args__ if isinstance(tp, (typing.TypeVar, ParamSpec))
        )

    # 3.9 used by __getitem__ below
    def copy_with(self, params):
        if isinstance(params[-1], _ConcatenateGenericAlias):
            params = (*params[:-1], *params[-1].__args__)
        elif isinstance(params[-1], (list, tuple)):
```

```
        return (*params[:-1], *params[-1])
    elif (not (params[-1] is ... or isinstance(params[-1], ParamSpec))):
        raise TypeError("The last parameter to Concatenate should be a "
                "ParamSpec variable or ellipsis.")
    return self.__class__(self.__origin__, params)

# 3.9; accessed during GenericAlias.__getitem__ when substituting
def __getitem__(self, args):
    if self.__origin__ in (Generic, Protocol):
        # Can't subscript Generic[...] or Protocol[...].
        raise TypeError(f"Cannot subscript already-subscripted {self}")
    if not self.__parameters__:
        raise TypeError(f"{self} is not a generic class")

    if not isinstance(args, tuple):
        args = (args,)
    args = _unpack_args(*(_type_convert(p) for p in args))
    params = self.__parameters__
    for param in params:
        prepare = getattr(param, "__typing_prepare_subst__", None)
        if prepare is not None:
            args = prepare(self, args)
        # 3.9 & typing.ParamSpec
        elif isinstance(param, ParamSpec):
            i = params.index(param)
            if (
                i == len(args)
                and getattr(param, '__default__', NoDefault) is not NoDefault
            ):
                args = [*args, param.__default__]
            if i >= len(args):
                raise TypeError(f"Too few arguments for {self}")
            # Special case for Z[[int, str, bool]] == Z[int, str, bool]
            if len(params) == 1 and not _is_param_expr(args[0]):
                assert i == 0
                args = (args,)
            elif (
                isinstance(args[i], list)
                # 3.9
                # This class inherits from list do not convert
                and not isinstance(args[i], _ConcatenateGenericAlias)
            ):
                args = (*args[:i], tuple(args[i]), *args[i + 1:])
```

```
            alen = len(args)
            plen = len(params)
            if alen != plen:
                raise TypeError(
                    f"Too {'many' if alen > plen else 'few'} arguments for {self};"
                    f" actual {alen}, expected {plen}"
                )

            subst = dict(zip(self.__parameters__, args))
            # determine new args
            new_args = []
            for arg in self.__args__:
                if isinstance(arg, type):
                    new_args.append(arg)
                    continue
                if isinstance(arg, TypeVar):
                    arg = subst[arg]
                    if (
                        (isinstance(arg, typing._GenericAlias) and _is_unpack(arg))
                        or (
                            hasattr(_types, "GenericAlias")
                            and isinstance(arg, _types.GenericAlias)
                            and getattr(arg, "__unpacked__", False)
                        )
                    ):
                        raise TypeError(f"{arg} is not valid as type argument")

                elif isinstance(arg,
                    typing._GenericAlias
                    if not hasattr(_types, "GenericAlias") else
                    (typing._GenericAlias, _types.GenericAlias)
                ):
                    subparams = arg.__parameters__
                    if subparams:
                        subargs = tuple(subst[x] for x in subparams)
                        arg = arg[subargs]
                new_args.append(arg)
            return self.copy_with(tuple(new_args))

    # 3.10+
    else:
        _ConcatenateGenericAlias = typing._ConcatenateGenericAlias

        # 3.10
```

```
    if sys.version_info < (3, 11):

        class _ConcatenateGenericAlias(typing._ConcatenateGenericAlias, _root=True):
            # needed for checks in collections.abc.Callable to accept this class
            __module__ = "typing"

            def copy_with(self, params):
                if isinstance(params[-1], (list, tuple)):
                    return (*params[:-1], *params[-1])
                if isinstance(params[-1], typing._ConcatenateGenericAlias):
                    params = (*params[:-1], *params[-1].__args__)
                elif not (params[-1] is ... or isinstance(params[-1], ParamSpec)):
                    raise TypeError("The last parameter to Concatenate should be a "
                            "ParamSpec variable or ellipsis.")
                return super(typing._ConcatenateGenericAlias, self).copy_with(params)

            def __getitem__(self, args):
                value = super().__getitem__(args)
                if isinstance(value, tuple) and any(_is_unpack(t) for t in value):
                    return tuple(_unpack_args(*(n for n in value)))
                return value


# 3.9.2
class _EllipsisDummy: ...


# <=3.10
def _create_concatenate_alias(origin, parameters):
    if parameters[-1] is ... and sys.version_info < (3, 9, 2):
        # Hack: Arguments must be types, replace it with one.
        parameters = (*parameters[:-1], _EllipsisDummy)
    if sys.version_info >= (3, 10, 3):
        concatenate = _ConcatenateGenericAlias(origin, parameters,
                                _typevar_types=(TypeVar, ParamSpec),
                                _paramspec_tvars=True)
    else:
        concatenate = _ConcatenateGenericAlias(origin, parameters)
    if parameters[-1] is not _EllipsisDummy:
        return concatenate
    # Remove dummy again
    concatenate.__args__ = tuple(p if p is not _EllipsisDummy else ...
                                 for p in concatenate.__args__)
    if sys.version_info < (3, 10):
```

```
        # backport needs __args__ adjustment only
        return concatenate
    concatenate.__parameters__ = tuple(p for p in concatenate.__parameters__
                                       if p is not _EllipsisDummy)
    return concatenate


# <=3.10
@typing._tp_cache
def _concatenate_getitem(self, parameters):
    if parameters == ():
        raise TypeError("Cannot take a Concatenate of no types.")
    if not isinstance(parameters, tuple):
        parameters = (parameters,)
    if not (parameters[-1] is ... or isinstance(parameters[-1], ParamSpec)):
        raise TypeError("The last parameter to Concatenate should be a "
                        "ParamSpec variable or ellipsis.")
    msg = "Concatenate[arg, ...]: each arg must be a type."
    parameters = (*(typing._type_check(p, msg) for p in parameters[:-1]),
                  parameters[-1])
    return _create_concatenate_alias(self, parameters)


# 3.11+; Concatenate does not accept ellipsis in 3.10
# Breakpoint: https://github.com/python/cpython/pull/30969
if sys.version_info >= (3, 11):
    Concatenate = typing.Concatenate
# <=3.10
else:
    @_ExtensionsSpecialForm
    def Concatenate(self, parameters):
        """Used in conjunction with ``ParamSpec`` and ``Callable`` to represent a
        higher order function which adds, removes or transforms parameters of a
        callable.

        For example::

           Callable[Concatenate[int, P], int]

        See PEP 612 for detailed information.
        """
        return _concatenate_getitem(self, parameters)
```

```
# 3.10+
if hasattr(typing, 'TypeGuard'):
    TypeGuard = typing.TypeGuard
# 3.9
else:
    @_ExtensionsSpecialForm
    def TypeGuard(self, parameters):
        """Special typing form used to annotate the return type of a user-defined
        type guard function.  ``TypeGuard`` only accepts a single type argument.
        At runtime, functions marked this way should return a boolean.

        ``TypeGuard`` aims to benefit *type narrowing* -- a technique used by static
        type checkers to determine a more precise type of an expression within a
        program's code flow.  Usually type narrowing is done by analyzing
        conditional code flow and applying the narrowing to a block of code.  The
        conditional expression here is sometimes referred to as a "type guard".

        Sometimes it would be convenient to use a user-defined boolean function
        as a type guard.  Such a function should use ``TypeGuard[...]`` as its
        return type to alert static type checkers to this intention.

        Using  ``-> TypeGuard`` tells the static type checker that for a given
        function:

        1. The return value is a boolean.
        2. If the return value is ``True``, the type of its argument
        is the type inside ``TypeGuard``.

        For example::

            def is_str(val: Union[str, float]):
                # "isinstance" type guard
                if isinstance(val, str):
                    # Type of ``val`` is narrowed to ``str``
                    ...
                else:
                    # Else, type of ``val`` is narrowed to ``float``.
                    ...

        Strict type narrowing is not enforced -- ``TypeB`` need not be a narrower
        form of ``TypeA`` (it can even be a wider form) and this may lead to
        type-unsafe results.  The main reason is to allow for things like
        narrowing ``List[object]`` to ``List[str]`` even though the latter is not
        a subtype of the former, since ``List`` is invariant.  The responsibility of
```

```
        writing type-safe type guards is left to the user.

        ``TypeGuard`` also works with type variables.  For more information, see
        PEP 647 (User-Defined Type Guards).
        """
        item = typing._type_check(parameters, f'{self} accepts only a single type.')
        return typing._GenericAlias(self, (item,))


# 3.13+
if hasattr(typing, 'TypeIs'):
    TypeIs = typing.TypeIs
# <=3.12
else:
    @_ExtensionsSpecialForm
    def TypeIs(self, parameters):
        """Special typing form used to annotate the return type of a user-defined
        type narrower function.  ``TypeIs`` only accepts a single type argument.
        At runtime, functions marked this way should return a boolean.

        ``TypeIs`` aims to benefit *type narrowing* -- a technique used by static
        type checkers to determine a more precise type of an expression within a
        program's code flow.  Usually type narrowing is done by analyzing
        conditional code flow and applying the narrowing to a block of code.  The
        conditional expression here is sometimes referred to as a "type guard".

        Sometimes it would be convenient to use a user-defined boolean function
        as a type guard.  Such a function should use ``TypeIs[...]`` as its
        return type to alert static type checkers to this intention.

        Using  ``-> TypeIs`` tells the static type checker that for a given
        function:

        1. The return value is a boolean.
        2. If the return value is ``True``, the type of its argument
        is the intersection of the type inside ``TypeIs`` and the argument's
        previously known type.

        For example::

            def is_awaitable(val: object) -> TypeIs[Awaitable[Any]]:
                return hasattr(val, '__await__')

            def f(val: Union[int, Awaitable[int]]) -> int:
```

```
            if is_awaitable(val):
                assert_type(val, Awaitable[int])
            else:
                assert_type(val, int)

        ``Typels`` also works with type variables.  For more information, see
        PEP 742 (Narrowing types with Typels).
        """
        item = typing._type_check(parameters, f'{self} accepts only a single type.')
        return typing._GenericAlias(self, (item,))


# 3.14+?
if hasattr(typing, 'TypeForm'):
    TypeForm = typing.TypeForm
# <=3.13
else:
    class _TypeFormForm(_ExtensionsSpecialForm, _root=True):
        # TypeForm(X) is equivalent to X but indicates to the type checker
        # that the object is a TypeForm.
        def __call__(self, obj, /):
            return obj

    @_TypeFormForm
    def TypeForm(self, parameters):
        """A special form representing the value that results from the evaluation
        of a type expression. This value encodes the information supplied in the
        type expression, and it represents the type described by that type expression.

        When used in a type expression, TypeForm describes a set of type form objects.
        It accepts a single type argument, which must be a valid type expression.
        ``TypeForm[T]`` describes the set of all type form objects that represent
        the type T or types that are assignable to T.

        Usage:

            def cast[T](typ: TypeForm[T], value: Any) -> T: ...

            reveal_type(cast(int, "x"))  # int

        See PEP 747 for more information.
        """
        item = typing._type_check(parameters, f'{self} accepts only a single type.')
        return typing._GenericAlias(self, (item,))
```

```
if hasattr(typing, "LiteralString"):  # 3.11+
    LiteralString = typing.LiteralString
else:
    @_SpecialForm
    def LiteralString(self, params):
        """Represents an arbitrary literal string.

        Example::

          from typing_extensions import LiteralString

          def query(sql: LiteralString) -> ...:
              ...

          query("SELECT * FROM table")  # ok
          query(f"SELECT * FROM {input()}")  # not ok

        See PEP 675 for details.

        """
        raise TypeError(f"{self} is not subscriptable")


if hasattr(typing, "Self"):  # 3.11+
    Self = typing.Self
else:
    @_SpecialForm
    def Self(self, params):
        """Used to spell the type of "self" in classes.

        Example::

          from typing import Self

          class ReturnsSelf:
              def parse(self, data: bytes) -> Self:
                  ...
                  return self

        """
```

```
        raise TypeError(f"{self} is not subscriptable")


if hasattr(typing, "Never"):  # 3.11+
    Never = typing.Never
else:
    @_SpecialForm
    def Never(self, params):
        """The bottom type, a type that has no members.

        This can be used to define a function that should never be
        called, or a function that never returns::

            from typing_extensions import Never

            def never_call_me(arg: Never) -> None:
                pass

            def int_or_str(arg: int | str) -> None:
                never_call_me(arg)  # type checker error
                match arg:
                    case int():
                        print("It's an int")
                    case str():
                        print("It's a str")
                    case _:
                        never_call_me(arg)  # ok, arg is of type Never

        """

        raise TypeError(f"{self} is not subscriptable")


if hasattr(typing, 'Required'):  # 3.11+
    Required = typing.Required
    NotRequired = typing.NotRequired
else:  # <=3.10
    @_ExtensionsSpecialForm
    def Required(self, parameters):
        """A special typing construct to mark a key of a total=False TypedDict
        as required. For example:

            class Movie(TypedDict, total=False):
```

```
            title: Required[str]
            year: int

        m = Movie(
            title='The Matrix',  # typechecker error if key is omitted
            year=1999,
        )

    There is no runtime checking that a required key is actually provided
    when instantiating a related TypedDict.
    """
    item = typing._type_check(parameters, f'{self._name} accepts only a single type.')
    return typing._GenericAlias(self, (item,))

@_ExtensionsSpecialForm
def NotRequired(self, parameters):
    """A special typing construct to mark a key of a TypedDict as
    potentially missing. For example:

        class Movie(TypedDict):
            title: str
            year: NotRequired[int]

        m = Movie(
            title='The Matrix',  # typechecker error if key is omitted
            year=1999,
        )
    """
    item = typing._type_check(parameters, f'{self._name} accepts only a single type.')
    return typing._GenericAlias(self, (item,))


if hasattr(typing, 'ReadOnly'):
    ReadOnly = typing.ReadOnly
else:  # <=3.12
    @_ExtensionsSpecialForm
    def ReadOnly(self, parameters):
        """A special typing construct to mark an item of a TypedDict as read-only.

        For example:

            class Movie(TypedDict):
                title: ReadOnly[str]
                year: int
```

```
        def mutate_movie(m: Movie) -> None:
            m["year"] = 1992  # allowed
            m["title"] = "The Matrix"  # typechecker error

        There is no runtime checking for this property.
        """
        item = typing._type_check(parameters, f'{self._name} accepts only a single type.')
        return typing._GenericAlias(self, (item,))


_UNPACK_DOC = """\
Type unpack operator.

The type unpack operator takes the child types from some container type,
such as `tuple[int, str]` or a `TypeVarTuple`, and 'pulls them out'. For
example:

  # For some generic class `Foo`:
  Foo[Unpack[tuple[int, str]]]  # Equivalent to Foo[int, str]

  Ts = TypeVarTuple('Ts')
  # Specifies that `Bar` is generic in an arbitrary number of types.
  # (Think of `Ts` as a tuple of an arbitrary number of individual
  #  `TypeVar`s, which the `Unpack` is 'pulling out' directly into the
  #  `Generic[]`.)
  class Bar(Generic[Unpack[Ts]]): ...
  Bar[int]  # Valid
  Bar[int, str]  # Also valid

From Python 3.11, this can also be done using the `*` operator:

    Foo[*tuple[int, str]]
    class Bar(Generic[*Ts]): ...

The operator can also be used along with a `TypedDict` to annotate
`**kwargs` in a function signature. For instance:

  class Movie(TypedDict):
    name: str
    year: int

  # This function expects two keyword arguments - *name* of type `str` and
  # *year* of type `int`.
```

```
  def foo(**kwargs: Unpack[Movie]): ...

Note that there is only some runtime checking of this operator. Not
everything the runtime allows may be accepted by static type checkers.

For more information, see PEP 646 and PEP 692.
"""


# PEP 692 changed the repr of Unpack[]
# Breakpoint: https://github.com/python/cpython/pull/104048
if sys.version_info >= (3, 12):
    Unpack = typing.Unpack

    def _is_unpack(obj):
        return get_origin(obj) is Unpack

else:  # <=3.11
    class _UnpackSpecialForm(_ExtensionsSpecialForm, _root=True):
        def __init__(self, getitem):
            super().__init__(getitem)
            self.__doc__ = _UNPACK_DOC

    class _UnpackAlias(typing._GenericAlias, _root=True):
        if sys.version_info < (3, 11):
            # needed for compatibility with Generic[Unpack[Ts]]
            __class__ = typing.TypeVar

        @property
        def __typing_unpacked_tuple_args__(self):
            assert self.__origin__ is Unpack
            assert len(self.__args__) == 1
            arg, = self.__args__
            if isinstance(arg, (typing._GenericAlias, _types.GenericAlias)):
                if arg.__origin__ is not tuple:
                    raise TypeError("Unpack[...] must be used with a tuple type")
                return arg.__args__
            return None

        @property
        def __typing_is_unpacked_typevartuple__(self):
            assert self.__origin__ is Unpack
            assert len(self.__args__) == 1
            return isinstance(self.__args__[0], TypeVarTuple)
```

```
        def __getitem__(self, args):
            if self.__typing_is_unpacked_typevartuple__:
                return args
            return super().__getitem__(args)

    @_UnpackSpecialForm
    def Unpack(self, parameters):
        item = typing._type_check(parameters, f'{self._name} accepts only a single type.')
        return _UnpackAlias(self, (item,))

    def _is_unpack(obj):
        return isinstance(obj, _UnpackAlias)


def _unpack_args(*args):
    newargs = []
    for arg in args:
        subargs = getattr(arg, '__typing_unpacked_tuple_args__', None)
        if subargs is not None and (not (subargs and subargs[-1] is ...)):
            newargs.extend(subargs)
        else:
            newargs.append(arg)
    return newargs


if _PEP_696_IMPLEMENTED:
    from typing import TypeVarTuple

elif hasattr(typing, "TypeVarTuple"):  # 3.11+

    # Add default parameter - PEP 696
    class TypeVarTuple(metaclass=_TypeVarLikeMeta):
        """Type variable tuple."""

        _backported_typevarlike = typing.TypeVarTuple

        def __new__(cls, name, *, default=NoDefault):
            tvt = typing.TypeVarTuple(name)
            _set_default(tvt, default)
            _set_module(tvt)

            def _typevartuple_prepare_subst(alias, args):
                params = alias.__parameters__
```

```
    typevartuple_index = params.index(tvt)
    for param in params[typevartuple_index + 1:]:
        if isinstance(param, TypeVarTuple):
            raise TypeError(
                f"More than one TypeVarTuple parameter in {alias}"
            )

    alen = len(args)
    plen = len(params)
    left = typevartuple_index
    right = plen - typevartuple_index - 1
    var_tuple_index = None
    fillarg = None
    for k, arg in enumerate(args):
        if not isinstance(arg, type):
            subargs = getattr(arg, '__typing_unpacked_tuple_args__', None)
            if subargs and len(subargs) == 2 and subargs[-1] is ...:
                if var_tuple_index is not None:
                    raise TypeError(
                        "More than one unpacked "
                        "arbitrary-length tuple argument"
                    )
                var_tuple_index = k
                fillarg = subargs[0]
    if var_tuple_index is not None:
        left = min(left, var_tuple_index)
        right = min(right, alen - var_tuple_index - 1)
    elif left + right > alen:
        raise TypeError(f"Too few arguments for {alias};"
                        f" actual {alen}, expected at least {plen - 1}")
    if left == alen - right and tvt.has_default():
        replacement = _unpack_args(tvt.__default__)
    else:
        replacement = args[left: alen - right]

    return (
        *args[:left],
        *([fillarg] * (typevartuple_index - left)),
        replacement,
        *([fillarg] * (plen - right - left - typevartuple_index - 1)),
        *args[alen - right:],
    )

tvt.__typing_prepare_subst__ = _typevartuple_prepare_subst
```

```
            return tvt

        def __init_subclass__(self, *args, **kwds):
            raise TypeError("Cannot subclass special typing classes")

else:  # <=3.10
    class TypeVarTuple(_DefaultMixin):
        """Type variable tuple.

        Usage::

            Ts = TypeVarTuple('Ts')

        In the same way that a normal type variable is a stand-in for a single
        type such as ``int``, a type variable *tuple* is a stand-in for a *tuple*
        type such as ``Tuple[int, str]``.

        Type variable tuples can be used in ``Generic`` declarations.
        Consider the following example::

            class Array(Generic[*Ts]): ...

        The ``Ts`` type variable tuple here behaves like ``tuple[T1, T2]``,
        where ``T1`` and ``T2`` are type variables. To use these type variables
        as type parameters of ``Array``, we must *unpack* the type variable tuple using
        the star operator: ``*Ts``. The signature of ``Array`` then behaves
        as if we had simply written ``class Array(Generic[T1, T2]): ...``.
        In contrast to ``Generic[T1, T2]``, however, ``Generic[*Shape]`` allows
        us to parameterise the class with an *arbitrary* number of type parameters.

        Type variable tuples can be used anywhere a normal ``TypeVar`` can.
        This includes class definitions, as shown above, as well as function
        signatures and variable annotations::

            class Array(Generic[*Ts]):

                def __init__(self, shape: Tuple[*Ts]):
                    self._shape: Tuple[*Ts] = shape

                def get_shape(self) -> Tuple[*Ts]:
                    return self._shape

            shape = (Height(480), Width(640))
            x: Array[Height, Width] = Array(shape)
```

```
            y = abs(x)  # Inferred type is Array[Height, Width]
            z = x + x   #       ...    is Array[Height, Width]
            x.get_shape()  #    ...    is tuple[Height, Width]

        """

        # Trick Generic __parameters__.
        __class__ = typing.TypeVar

        def __iter__(self):
            yield self.__unpacked__

        def __init__(self, name, *, default=NoDefault):
            self.__name__ = name
            _DefaultMixin.__init__(self, default)

            # for pickling:
            def_mod = _caller()
            if def_mod != 'typing_extensions':
                self.__module__ = def_mod

            self.__unpacked__ = Unpack[self]

        def __repr__(self):
            return self.__name__

        def __hash__(self):
            return object.__hash__(self)

        def __eq__(self, other):
            return self is other

        def __reduce__(self):
            return self.__name__

        def __init_subclass__(self, *args, **kwds):
            if '_root' not in kwds:
                raise TypeError("Cannot subclass special typing classes")


if hasattr(typing, "reveal_type"):  # 3.11+
    reveal_type = typing.reveal_type
else:  # <=3.10
    def reveal_type(obj: T, /) -> T:
```

```
        """Reveal the inferred type of a variable.

        When a static type checker encounters a call to ``reveal_type()``,
        it will emit the inferred type of the argument::

            x: int = 1
            reveal_type(x)

        Running a static type checker (e.g., ``mypy``) on this example
        will produce output similar to 'Revealed type is "builtins.int"'.

        At runtime, the function prints the runtime type of the
        argument and returns it unchanged.

        """
        print(f"Runtime type is {type(obj).__name__!r}", file=sys.stderr)
        return obj


if hasattr(typing, "_ASSERT_NEVER_REPR_MAX_LENGTH"):  # 3.11+
    _ASSERT_NEVER_REPR_MAX_LENGTH = typing._ASSERT_NEVER_REPR_MAX_LENGTH
else:  # <=3.10
    _ASSERT_NEVER_REPR_MAX_LENGTH = 100


if hasattr(typing, "assert_never"):  # 3.11+
    assert_never = typing.assert_never
else:  # <=3.10
    def assert_never(arg: Never, /) -> Never:
        """Assert to the type checker that a line of code is unreachable.

        Example::

            def int_or_str(arg: int | str) -> None:
                match arg:
                    case int():
                        print("It's an int")
                    case str():
                        print("It's a str")
                    case _:
                        assert_never(arg)

        If a type checker finds that a call to assert_never() is
        reachable, it will emit an error.
```

```
        At runtime, this throws an exception when called.

        """
        value = repr(arg)
        if len(value) > _ASSERT_NEVER_REPR_MAX_LENGTH:
            value = value[:_ASSERT_NEVER_REPR_MAX_LENGTH] + '...'
        raise AssertionError(f"Expected code to be unreachable, but got: {value}")


# dataclass_transform exists in 3.11 but lacks the frozen_default parameter
# Breakpoint: https://github.com/python/cpython/pull/99958
if sys.version_info >= (3, 12):  # 3.12+
    dataclass_transform = typing.dataclass_transform
else:  # <=3.11
    def dataclass_transform(
        *,
        eq_default: bool = True,
        order_default: bool = False,
        kw_only_default: bool = False,
        frozen_default: bool = False,
        field_specifiers: typing.Tuple[
            typing.Union[typing.Type[typing.Any], typing.Callable[..., typing.Any]],
            ...
        ] = (),
        **kwargs: typing.Any,
    ) -> typing.Callable[[T], T]:
        """Decorator that marks a function, class, or metaclass as providing
        dataclass-like behavior.

        Example:

            from typing_extensions import dataclass_transform

            _T = TypeVar("_T")

            # Used on a decorator function
            @dataclass_transform()
            def create_model(cls: type[_T]) -> type[_T]:
                ...
                return cls

            @create_model
            class CustomerModel:
```

```
    id: int
    name: str

# Used on a base class
@dataclass_transform()
class ModelBase: ...

class CustomerModel(ModelBase):
    id: int
    name: str

# Used on a metaclass
@dataclass_transform()
class ModelMeta(type): ...

class ModelBase(metaclass=ModelMeta): ...

class CustomerModel(ModelBase):
    id: int
    name: str
```

```
Each of the ``CustomerModel`` classes defined in this example will now
behave similarly to a dataclass created with the ``@dataclasses.dataclass``
decorator. For example, the type checker will synthesize an ``__init__``
method.

The arguments to this decorator can be used to customize this behavior:
- ``eq_default`` indicates whether the ``eq`` parameter is assumed to be
  True or False if it is omitted by the caller.
- ``order_default`` indicates whether the ``order`` parameter is
  assumed to be True or False if it is omitted by the caller.
- ``kw_only_default`` indicates whether the ``kw_only`` parameter is
  assumed to be True or False if it is omitted by the caller.
- ``frozen_default`` indicates whether the ``frozen`` parameter is
  assumed to be True or False if it is omitted by the caller.
- ``field_specifiers`` specifies a static list of supported classes
  or functions that describe fields, similar to ``dataclasses.field()``.

At runtime, this decorator records its arguments in the
``__dataclass_transform__`` attribute on the decorated object.

See PEP 681 for details.

"""
```

```
        def decorator(cls_or_fn):
            cls_or_fn.__dataclass_transform__ = {
                "eq_default": eq_default,
                "order_default": order_default,
                "kw_only_default": kw_only_default,
                "frozen_default": frozen_default,
                "field_specifiers": field_specifiers,
                "kwargs": kwargs,
            }
            return cls_or_fn
        return decorator


if hasattr(typing, "override"):  # 3.12+
    override = typing.override
else:  # <=3.11
    _F = typing.TypeVar("_F", bound=typing.Callable[..., typing.Any])

    def override(arg: _F, /) -> _F:
        """Indicate that a method is intended to override a method in a base class.

        Usage:

            class Base:
                def method(self) -> None:
                    pass

            class Child(Base):
                @override
                def method(self) -> None:
                    super().method()

        When this decorator is applied to a method, the type checker will
        validate that it overrides a method with the same name on a base class.
        This helps prevent bugs that may occur when a base class is changed
        without an equivalent change to a child class.

        There is no runtime checking of these properties. The decorator
        sets the ``__override__`` attribute to ``True`` on the decorated object
        to allow runtime introspection.

        See PEP 698 for details.

        """
```

```
        try:
            arg.__override__ = True
        except (AttributeError, TypeError):
            # Skip the attribute silently if it is not writable.
            # AttributeError happens if the object has __slots__ or a
            # read-only property, TypeError if it's a builtin class.
            pass
        return arg


# Python 3.13.3+ contains a fix for the wrapped __new__
# Breakpoint: https://github.com/python/cpython/pull/132160
if sys.version_info >= (3, 13, 3):
    deprecated = warnings.deprecated
else:
    _T = typing.TypeVar("_T")

    class deprecated:
        """Indicate that a class, function or overload is deprecated.

        When this decorator is applied to an object, the type checker
        will generate a diagnostic on usage of the deprecated object.

        Usage:

            @deprecated("Use B instead")
            class A:
                pass

            @deprecated("Use g instead")
            def f():
                pass

            @overload
            @deprecated("int support is deprecated")
            def g(x: int) -> int: ...
            @overload
            def g(x: str) -> int: ...

        The warning specified by *category* will be emitted at runtime
        on use of deprecated objects. For functions, that happens on calls;
        for classes, on instantiation and on creation of subclasses.
        If the *category* is ``None``, no warning is emitted at runtime.
        The *stacklevel* determines where the
```

warning is emitted. If it is ``1`` (the default), the warning
is emitted at the direct caller of the deprecated object; if it
is higher, it is emitted further up the stack.
*Static type checker behavior is not affected by the *category**
*and *stacklevel* arguments.*

The deprecation message passed to the decorator is saved in the
``__deprecated__`` attribute on the decorated object.
If applied to an overload, the decorator
must be after the ``@overload`` decorator for the attribute to
exist on the overload as returned by ``get_overloads()``.

See PEP 702 for details.

"""
```
def __init__(
    self,
    message: str,
    /,
    *,
    category: typing.Optional[typing.Type[Warning]] = DeprecationWarning,
    stacklevel: int = 1,
) -> None:
    if not isinstance(message, str):
        raise TypeError(
            "Expected an object of type str for 'message', not "
            f"{type(message).__name__!r}"
        )
    self.message = message
    self.category = category
    self.stacklevel = stacklevel

def __call__(self, arg: _T, /) -> _T:
    # Make sure the inner functions created below don't
    # retain a reference to self.
    msg = self.message
    category = self.category
    stacklevel = self.stacklevel
    if category is None:
        arg.__deprecated__ = msg
        return arg
    elif isinstance(arg, type):
        import functools
        from types import MethodType
```

```
    original_new = arg.__new__

    @functools.wraps(original_new)
    def __new__(cls, /, *args, **kwargs):
        if cls is arg:
            warnings.warn(msg, category=category, stacklevel=stacklevel + 1)
        if original_new is not object.__new__:
            return original_new(cls, *args, **kwargs)
        # Mirrors a similar check in object.__new__.
        elif cls.__init__ is object.__init__ and (args or kwargs):
            raise TypeError(f"{cls.__name__}() takes no arguments")
        else:
            return original_new(cls)

    arg.__new__ = staticmethod(__new__)

    original_init_subclass = arg.__init_subclass__
    # We need slightly different behavior if __init_subclass__
    # is a bound method (likely if it was implemented in Python)
    if isinstance(original_init_subclass, MethodType):
        original_init_subclass = original_init_subclass.__func__

        @functools.wraps(original_init_subclass)
        def __init_subclass__(*args, **kwargs):
            warnings.warn(msg, category=category, stacklevel=stacklevel + 1)
            return original_init_subclass(*args, **kwargs)

        arg.__init_subclass__ = classmethod(__init_subclass__)
    # Or otherwise, which likely means it's a builtin such as
    # object's implementation of __init_subclass__.
    else:
        @functools.wraps(original_init_subclass)
        def __init_subclass__(*args, **kwargs):
            warnings.warn(msg, category=category, stacklevel=stacklevel + 1)
            return original_init_subclass(*args, **kwargs)

        arg.__init_subclass__ = __init_subclass__

    arg.__deprecated__ = __new__.__deprecated__ = msg
    __init_subclass__.__deprecated__ = msg
    return arg
elif callable(arg):
    import asyncio.coroutines
```

```
            import functools
            import inspect

            @functools.wraps(arg)
            def wrapper(*args, **kwargs):
                warnings.warn(msg, category=category, stacklevel=stacklevel + 1)
                return arg(*args, **kwargs)

            if asyncio.coroutines.iscoroutinefunction(arg):
                # Breakpoint: https://github.com/python/cpython/pull/99247
                if sys.version_info >= (3, 12):
                    wrapper = inspect.markcoroutinefunction(wrapper)
                else:
                    wrapper._is_coroutine = asyncio.coroutines._is_coroutine

            arg.__deprecated__ = wrapper.__deprecated__ = msg
            return wrapper
        else:
            raise TypeError(
                "@deprecated decorator with non-None category must be applied to "
                f"a class or callable, not {arg!r}"
            )

# Breakpoint: https://github.com/python/cpython/pull/23702
if sys.version_info < (3, 10):
    def _is_param_expr(arg):
        return arg is ... or isinstance(
            arg, (tuple, list, ParamSpec, _ConcatenateGenericAlias)
        )
else:
    def _is_param_expr(arg):
        return arg is ... or isinstance(
            arg,
            (
                tuple,
                list,
                ParamSpec,
                _ConcatenateGenericAlias,
                typing._ConcatenateGenericAlias,
            ),
        )


# We have to do some monkey patching to deal with the dual nature of
```

```
# Unpack/TypeVarTuple:
# - We want Unpack to be a kind of TypeVar so it gets accepted in
#   Generic[Unpack[Ts]]
# - We want it to *not* be treated as a TypeVar for the purposes of
#   counting generic parameters, so that when we subscript a generic,
#   the runtime doesn't try to substitute the Unpack with the subscripted type.
if not hasattr(typing, "TypeVarTuple"):
    def _check_generic(cls, parameters, elen=_marker):
        """Check correct count for parameters of a generic cls (internal helper).

        This gives a nice error message in case of count mismatch.
        """
        # If substituting a single ParamSpec with multiple arguments
        # we do not check the count
        if (inspect.isclass(cls) and issubclass(cls, typing.Generic)
            and len(cls.__parameters__) == 1
            and isinstance(cls.__parameters__[0], ParamSpec)
            and parameters
            and not _is_param_expr(parameters[0])
        ):
            # Generic modifies parameters variable, but here we cannot do this
            return

        if not elen:
            raise TypeError(f"{cls} is not a generic class")
        if elen is _marker:
            if not hasattr(cls, "__parameters__") or not cls.__parameters__:
                raise TypeError(f"{cls} is not a generic class")
            elen = len(cls.__parameters__)
        alen = len(parameters)
        if alen != elen:
            expect_val = elen
            if hasattr(cls, "__parameters__"):
                parameters = [p for p in cls.__parameters__ if not _is_unpack(p)]
                num_tv_tuples = sum(isinstance(p, TypeVarTuple) for p in parameters)
                if (num_tv_tuples > 0) and (alen >= elen - num_tv_tuples):
                    return

                # deal with TypeVarLike defaults
                # required TypeVarLikes cannot appear after a defaulted one.
                if alen < elen:
                    # since we validate TypeVarLike default in _collect_type_vars
                    # or _collect_parameters we can safely check parameters[alen]
                    if (
```

```
                    getattr(parameters[alen], '__default__', NoDefault)
                    is not NoDefault
                ):
                    return

                num_default_tv = sum(getattr(p, '__default__', NoDefault)
                                     is not NoDefault for p in parameters)

                elen -= num_default_tv

                expect_val = f"at least {elen}"

            # Breakpoint: https://github.com/python/cpython/pull/27515
            things = "arguments" if sys.version_info >= (3, 10) else "parameters"
            raise TypeError(f"Too {'many' if alen > elen else 'few'} {things}"
                            f" for {cls}; actual {alen}, expected {expect_val}")
    else:
        # Python 3.11+

        def _check_generic(cls, parameters, elen):
            """Check correct count for parameters of a generic cls (internal helper).

            This gives a nice error message in case of count mismatch.
            """
            if not elen:
                raise TypeError(f"{cls} is not a generic class")
            alen = len(parameters)
            if alen != elen:
                expect_val = elen
                if hasattr(cls, "__parameters__"):
                    parameters = [p for p in cls.__parameters__ if not _is_unpack(p)]

                    # deal with TypeVarLike defaults
                    # required TypeVarLikes cannot appear after a defaulted one.
                    if alen < elen:
                        # since we validate TypeVarLike default in _collect_type_vars
                        # or _collect_parameters we can safely check parameters[alen]
                        if (
                            getattr(parameters[alen], '__default__', NoDefault)
                            is not NoDefault
                        ):
                            return

                        num_default_tv = sum(getattr(p, '__default__', NoDefault)
```

```
                                is not NoDefault for p in parameters)

                elen -= num_default_tv

                expect_val = f"at least {elen}"

            raise TypeError(f"Too {'many' if alen > elen else 'few'} arguments"
                            f" for {cls}; actual {alen}, expected {expect_val}")

if not _PEP_696_IMPLEMENTED:
    typing._check_generic = _check_generic


def _has_generic_or_protocol_as_origin() -> bool:
    try:
        frame = sys._getframe(2)
    # - Catch AttributeError: not all Python implementations have sys._getframe()
    # - Catch ValueError: maybe we're called from an unexpected module
    #   and the call stack isn't deep enough
    except (AttributeError, ValueError):
        return False  # err on the side of leniency
    else:
        # If we somehow get invoked from outside typing.py,
        # also err on the side of leniency
        if frame.f_globals.get("__name__") != "typing":
            return False
        origin = frame.f_locals.get("origin")
        # Cannot use "in" because origin may be an object with a buggy __eq__ that
        # throws an error.
        return origin is typing.Generic or origin is Protocol or origin is typing.Protocol


_TYPEVARTUPLE_TYPES = {TypeVarTuple, getattr(typing, "TypeVarTuple", None)}


def _is_unpacked_typevartuple(x) -> bool:
    if get_origin(x) is not Unpack:
        return False
    args = get_args(x)
    return (
        bool(args)
        and len(args) == 1
        and type(args[0]) in _TYPEVARTUPLE_TYPES
    )
```

```
# Python 3.11+ _collect_type_vars was renamed to _collect_parameters
if hasattr(typing, '_collect_type_vars'):
    def _collect_type_vars(types, typevar_types=None):
        """Collect all type variable contained in types in order of
        first appearance (lexicographic order). For example::

          _collect_type_vars((T, List[S, T])) == (T, S)
        """
        if typevar_types is None:
            typevar_types = typing.TypeVar
        tvars = []

        # A required TypeVarLike cannot appear after a TypeVarLike with a default
        # if it was a direct call to `Generic[]` or `Protocol[]`
        enforce_default_ordering = _has_generic_or_protocol_as_origin()
        default_encountered = False

        # Also, a TypeVarLike with a default cannot appear after a TypeVarTuple
        type_var_tuple_encountered = False

        for t in types:
            if _is_unpacked_typevartuple(t):
                type_var_tuple_encountered = True
            elif (
                isinstance(t, typevar_types) and not isinstance(t, _UnpackAlias)
                and t not in tvars
            ):
                if enforce_default_ordering:
                    has_default = getattr(t, '__default__', NoDefault) is not NoDefault
                    if has_default:
                        if type_var_tuple_encountered:
                            raise TypeError('Type parameter with a default'
                                            ' follows TypeVarTuple')
                        default_encountered = True
                    elif default_encountered:
                        raise TypeError(f'Type parameter {t!r} without a default'
                                        ' follows type parameter with a default')

                tvars.append(t)
            if _should_collect_from_parameters(t):
                tvars.extend([t for t in t.__parameters__ if t not in tvars])
            elif isinstance(t, tuple):
```

```
                # Collect nested type_vars
                # tuple wrapped by  _prepare_paramspec_params(cls, params)
                for x in t:
                    for collected in _collect_type_vars([x]):
                        if collected not in tvars:
                            tvars.append(collected)
        return tuple(tvars)

    typing._collect_type_vars = _collect_type_vars
else:
    def _collect_parameters(args):
        """Collect all type variables and parameter specifications in args
        in order of first appearance (lexicographic order).

        For example::

            assert _collect_parameters((T, Callable[P, T])) == (T, P)
        """
        parameters = []

        # A required TypeVarLike cannot appear after a TypeVarLike with default
        # if it was a direct call to `Generic[]` or `Protocol[]`
        enforce_default_ordering = _has_generic_or_protocol_as_origin()
        default_encountered = False

        # Also, a TypeVarLike with a default cannot appear after a TypeVarTuple
        type_var_tuple_encountered = False

        for t in args:
            if isinstance(t, type):
                # We don't want __parameters__ descriptor of a bare Python class.
                pass
            elif isinstance(t, tuple):
                # `t` might be a tuple, when `ParamSpec` is substituted with
                # `[T, int]`, or `[int, *Ts]`, etc.
                for x in t:
                    for collected in _collect_parameters([x]):
                        if collected not in parameters:
                            parameters.append(collected)
            elif hasattr(t, '__typing_subst__'):
                if t not in parameters:
                    if enforce_default_ordering:
                        has_default = (
                            getattr(t, '__default__', NoDefault) is not NoDefault
```

```
                    )

                    if type_var_tuple_encountered and has_default:
                        raise TypeError('Type parameter with a default'
                                        ' follows TypeVarTuple')

                    if has_default:
                        default_encountered = True
                    elif default_encountered:
                        raise TypeError(f'Type parameter {t!r} without a default'
                                        ' follows type parameter with a default')

                    parameters.append(t)
                else:
                    if _is_unpacked_typevartuple(t):
                        type_var_tuple_encountered = True
                    for x in getattr(t, '__parameters__', ()):
                        if x not in parameters:
                            parameters.append(x)

            return tuple(parameters)

        if not _PEP_696_IMPLEMENTED:
            typing._collect_parameters = _collect_parameters

    # Backport typing.NamedTuple as it exists in Python 3.13.
    # In 3.11, the ability to define generic `NamedTuple`s was supported.
    # This was explicitly disallowed in 3.9-3.10, and only half-worked in <=3.8.
    # On 3.12, we added __orig_bases__ to call-based NamedTuples
    # On 3.13, we deprecated kwargs-based NamedTuples
    # Breakpoint: https://github.com/python/cpython/pull/105609
    if sys.version_info >= (3, 13):
        NamedTuple = typing.NamedTuple
    else:
        def _make_nmtuple(name, types, module, defaults=()):
            fields = [n for n, t in types]
            annotations = {n: typing._type_check(t, f"field {n} annotation must be a type")
                           for n, t in types}
            nm_tpl = collections.namedtuple(name, fields,
                                            defaults=defaults, module=module)
            nm_tpl.__annotations__ = nm_tpl.__new__.__annotations__ = annotations
            return nm_tpl

        _prohibited_namedtuple_fields = typing._prohibited
```

```
_special_namedtuple_fields = frozenset({'__module__', '__name__', '__annotations__'})

class _NamedTupleMeta(type):
    def __new__(cls, typename, bases, ns):
        assert _NamedTuple in bases
        for base in bases:
            if base is not _NamedTuple and base is not typing.Generic:
                raise TypeError(
                    'can only inherit from a NamedTuple type and Generic')
        bases = tuple(tuple if base is _NamedTuple else base for base in bases)
        if "__annotations__" in ns:
            types = ns["__annotations__"]
        elif "__annotate__" in ns:
            # TODO: Use inspect.VALUE here, and make the annotations lazily evaluated
            types = ns["__annotate__"](1)
        else:
            types = {}
        default_names = []
        for field_name in types:
            if field_name in ns:
                default_names.append(field_name)
            elif default_names:
                raise TypeError(f"Non-default namedtuple field {field_name} "
                                f"cannot follow default field"
                                f"{'s' if len(default_names) > 1 else ''} "
                                f"{', '.join(default_names)}")
        nm_tpl = _make_nmtuple(
            typename, types.items(),
            defaults=[ns[n] for n in default_names],
            module=ns['__module__']
        )
        nm_tpl.__bases__ = bases
        if typing.Generic in bases:
            if hasattr(typing, '_generic_class_getitem'):  # 3.12+
                nm_tpl.__class_getitem__ = classmethod(typing._generic_class_getitem)
            else:
                class_getitem = typing.Generic.__class_getitem__.__func__
                nm_tpl.__class_getitem__ = classmethod(class_getitem)
        # update from user namespace without overriding special namedtuple attributes
        for key, val in ns.items():
            if key in _prohibited_namedtuple_fields:
                raise AttributeError("Cannot overwrite NamedTuple attribute " + key)
            elif key not in _special_namedtuple_fields:
                if key not in nm_tpl._fields:
```

```
                setattr(nm_tpl, key, ns[key])
            try:
                set_name = type(val).__set_name__
            except AttributeError:
                pass
            else:
                try:
                    set_name(val, nm_tpl, key)
                except BaseException as e:
                    msg = (
                        f"Error calling __set_name__ on {type(val).__name__!r} "
                        f"instance {key!r} in {typename!r}"
                    )
                    # BaseException.add_note() existed on py311,
                    # but the __set_name__ machinery didn't start
                    # using add_note() until py312.
                    # Making sure exceptions are raised in the same way
                    # as in "normal" classes seems most important here.
                    # Breakpoint: https://github.com/python/cpython/pull/95915
                    if sys.version_info >= (3, 12):
                        e.add_note(msg)
                        raise
                    else:
                        raise RuntimeError(msg) from e

        if typing.Generic in bases:
            nm_tpl.__init_subclass__()
        return nm_tpl

_NamedTuple = type.__new__(_NamedTupleMeta, 'NamedTuple', (), {})

def _namedtuple_mro_entries(bases):
    assert NamedTuple in bases
    return (_NamedTuple,)

def NamedTuple(typename, fields=_marker, /, **kwargs):
    """Typed version of namedtuple.

    Usage::

        class Employee(NamedTuple):
            name: str
            id: int
```

```
*This is equivalent to::*

   *Employee = collections.namedtuple('Employee', ['name', 'id'])*

*The resulting class has an extra __annotations__ attribute, giving a
dict that maps field names to types.  (The field names are also in
the _fields attribute, which is part of the namedtuple API.)
An alternative equivalent functional syntax is also accepted::*

   *Employee = NamedTuple('Employee', [('name', str), ('id', int)])*
"""
if fields is _marker:
    if kwargs:
        deprecated_thing = "Creating NamedTuple classes using keyword arguments"
        deprecation_msg = (
            "{name} is deprecated and will be disallowed in Python {remove}. "
            "Use the class-based or functional syntax instead."
        )
    else:
        deprecated_thing = "Failing to pass a value for the 'fields' parameter"
        example = f"`{typename} = NamedTuple({typename!r}, [])`"
        deprecation_msg = (
            "{name} is deprecated and will be disallowed in Python {remove}. "
            "To create a NamedTuple class with 0 fields "
            "using the functional syntax, "
            "pass an empty list, e.g. "
        ) + example + "."
elif fields is None:
    if kwargs:
        raise TypeError(
            "Cannot pass `None` as the 'fields' parameter "
            "and also specify fields using keyword arguments"
        )
    else:
        deprecated_thing = "Passing `None` as the 'fields' parameter"
        example = f"`{typename} = NamedTuple({typename!r}, [])`"
        deprecation_msg = (
            "{name} is deprecated and will be disallowed in Python {remove}. "
            "To create a NamedTuple class with 0 fields "
            "using the functional syntax, "
            "pass an empty list, e.g. "
        ) + example + "."
elif kwargs:
    raise TypeError("Either list of fields or keywords"
```

```
                        " can be provided to NamedTuple, not both")
        if fields is _marker or fields is None:
            warnings.warn(
                deprecation_msg.format(name=deprecated_thing, remove="3.15"),
                DeprecationWarning,
                stacklevel=2,
            )
            fields = kwargs.items()
        nt = _make_nmtuple(typename, fields, module=_caller())
        nt.__orig_bases__ = (NamedTuple,)
        return nt

    NamedTuple.__mro_entries__ = _namedtuple_mro_entries


if hasattr(collections.abc, "Buffer"):
    Buffer = collections.abc.Buffer
else:
    class Buffer(abc.ABC):  # noqa: B024
        """Base class for classes that implement the buffer protocol.

        The buffer protocol allows Python objects to expose a low-level
        memory buffer interface. Before Python 3.12, it is not possible
        to implement the buffer protocol in pure Python code, or even
        to check whether a class implements the buffer protocol. In
        Python 3.12 and higher, the ``__buffer__`` method allows access
        to the buffer protocol from Python code, and the
        ``collections.abc.Buffer`` ABC allows checking whether a class
        implements the buffer protocol.

        To indicate support for the buffer protocol in earlier versions,
        inherit from this ABC, either in a stub file or at runtime,
        or use ABC registration. This ABC provides no methods, because
        there is no Python-accessible methods shared by pre-3.12 buffer
        classes. It is useful primarily for static checks.

        """

    # As a courtesy, register the most common stdlib buffer classes.
    Buffer.register(memoryview)
    Buffer.register(bytearray)
    Buffer.register(bytes)
```

```
# Backport of types.get_original_bases, available on 3.12+ in CPython
if hasattr(_types, "get_original_bases"):
    get_original_bases = _types.get_original_bases
else:
    def get_original_bases(cls, /):
        """Return the class's "original" bases prior to modification by `__mro_entries__`.

        Examples::

            from typing import TypeVar, Generic
            from typing_extensions import NamedTuple, TypedDict

            T = TypeVar("T")
            class Foo(Generic[T]): ...
            class Bar(Foo[int], float): ...
            class Baz(list[str]): ...
            Eggs = NamedTuple("Eggs", [("a", int), ("b", str)])
            Spam = TypedDict("Spam", {"a": int, "b": str})

            assert get_original_bases(Bar) == (Foo[int], float)
            assert get_original_bases(Baz) == (list[str],)
            assert get_original_bases(Eggs) == (NamedTuple,)
            assert get_original_bases(Spam) == (TypedDict,)
            assert get_original_bases(int) == (object,)
        """
        try:
            return cls.__dict__.get("__orig_bases__", cls.__bases__)
        except AttributeError:
            raise TypeError(
                f'Expected an instance of type, not {type(cls).__name__!r}'
            ) from None


# NewType is a class on Python 3.10+, making it pickleable
# The error message for subclassing instances of NewType was improved on 3.11+
# Breakpoint: https://github.com/python/cpython/pull/30268
if sys.version_info >= (3, 11):
    NewType = typing.NewType
else:
    class NewType:
        """NewType creates simple unique types with almost zero
        runtime overhead. NewType(name, tp) is considered a subtype of tp
        by static type checkers. At runtime, NewType(name, tp) returns
        a dummy callable that simply returns its argument. Usage::
```

```
    UserId = NewType('UserId', int)
    def name_by_id(user_id: UserId) -> str:
        ...
    UserId('user')          # Fails type check
    name_by_id(42)          # Fails type check
    name_by_id(UserId(42))  # OK
    num = UserId(5) + 1     # type: int
"""

def __call__(self, obj, /):
    return obj

def __init__(self, name, tp):
    self.__qualname__ = name
    if '.' in name:
        name = name.rpartition('.')[-1]
    self.__name__ = name
    self.__supertype__ = tp
    def_mod = _caller()
    if def_mod != 'typing_extensions':
        self.__module__ = def_mod

def __mro_entries__(self, bases):
    # We defined __mro_entries__ to get a better error message
    # if a user attempts to subclass a NewType instance. bpo-46170
    supercls_name = self.__name__

    class Dummy:
        def __init_subclass__(cls):
            subcls_name = cls.__name__
            raise TypeError(
                f"Cannot subclass an instance of NewType. "
                f"Perhaps you were looking for: "
                f"`{subcls_name} = NewType({subcls_name!r}, {supercls_name})`"
            )

    return (Dummy,)

def __repr__(self):
    return f'{self.__module__}.{self.__qualname__}'

def __reduce__(self):
    return self.__qualname__
```

```
        # Breakpoint: https://github.com/python/cpython/pull/21515
        if sys.version_info >= (3, 10):
            # PEP 604 methods
            # It doesn't make sense to have these methods on Python <3.10

            def __or__(self, other):
                return typing.Union[self, other]

            def __ror__(self, other):
                return typing.Union[other, self]


# Breakpoint: https://github.com/python/cpython/pull/124795
if sys.version_info >= (3, 14):
    TypeAliasType = typing.TypeAliasType
# <=3.13
else:
    # Breakpoint: https://github.com/python/cpython/pull/103764
    if sys.version_info >= (3, 12):
        # 3.12-3.13
        def _is_unionable(obj):
            """Corresponds to is_unionable() in unionobject.c in CPython."""
            return obj is None or isinstance(obj, (
                type,
                _types.GenericAlias,
                _types.UnionType,
                typing.TypeAliasType,
                TypeAliasType,
            ))
    else:
        # <=3.11
        def _is_unionable(obj):
            """Corresponds to is_unionable() in unionobject.c in CPython."""
            return obj is None or isinstance(obj, (
                type,
                _types.GenericAlias,
                _types.UnionType,
                TypeAliasType,
            ))

    if sys.version_info < (3, 10):
        # Copied and pasted from
https://github.com/python/cpython/blob/986a4e1b6fcae7fe7a1d0a26aea446107dd58dd2/
Objects/genericaliasobject.c#L568-L582,
```

```
    # so that we emulate the behaviour of `types.GenericAlias`
    # on the latest versions of CPython
    _ATTRIBUTE_DELEGATION_EXCLUSIONS = frozenset({
        "__class__",
        "__bases__",
        "__origin__",
        "__args__",
        "__unpacked__",
        "__parameters__",
        "__typing_unpacked_tuple_args__",
        "__mro_entries__",
        "__reduce_ex__",
        "__reduce__",
        "__copy__",
        "__deepcopy__",
    })

    class _TypeAliasGenericAlias(typing._GenericAlias, _root=True):
        def __getattr__(self, attr):
            if attr in _ATTRIBUTE_DELEGATION_EXCLUSIONS:
                return object.__getattr__(self, attr)
            return getattr(self.__origin__, attr)


class TypeAliasType:
    """Create named, parameterized type aliases.

    This provides a backport of the new `type` statement in Python 3.12:

        type ListOrSet[T] = list[T] | set[T]

    is equivalent to:

        T = TypeVar("T")
        ListOrSet = TypeAliasType("ListOrSet", list[T] | set[T], type_params=(T,))

    The name ListOrSet can then be used as an alias for the type it refers to.

    The type_params argument should contain all the type parameters used
    in the value of the type alias. If the alias is not generic, this
    argument is omitted.

    Static type checkers should only support type aliases declared using
    TypeAliasType that follow these rules:
```

- The first argument (the name) must be a string literal.
- The TypeAliasType instance must be immediately assigned to a variable of the same name. (For example, 'X = TypeAliasType("Y", int)' is invalid, as is 'X, Y = TypeAliasType("X", int), TypeAliasType("Y", int)').

"""

```
def __init__(self, name: str, value, *, type_params=()):
    if not isinstance(name, str):
        raise TypeError("TypeAliasType name must be a string")
    if not isinstance(type_params, tuple):
        raise TypeError("type_params must be a tuple")
    self.__value__ = value
    self.__type_params__ = type_params

    default_value_encountered = False
    parameters = []
    for type_param in type_params:
        if (
            not isinstance(type_param, (TypeVar, TypeVarTuple, ParamSpec))
            # <=3.11
            # Unpack Backport passes isinstance(type_param, TypeVar)
            or _is_unpack(type_param)
        ):
            raise TypeError(f"Expected a type param, got {type_param!r}")
        has_default = (
            getattr(type_param, '__default__', NoDefault) is not NoDefault
        )
        if default_value_encountered and not has_default:
            raise TypeError(f"non-default type parameter '{type_param!r}'"
                            " follows default type parameter")
        if has_default:
            default_value_encountered = True
        if isinstance(type_param, TypeVarTuple):
            parameters.extend(type_param)
        else:
            parameters.append(type_param)
    self.__parameters__ = tuple(parameters)
    def_mod = _caller()
    if def_mod != 'typing_extensions':
        self.__module__ = def_mod
    # Setting this attribute closes the TypeAliasType from further modification
    self.__name__ = name
```

```
def __setattr__(self, name: str, value: object, /) -> None:
    if hasattr(self, "__name__"):
        self._raise_attribute_error(name)
    super().__setattr__(name, value)

def __delattr__(self, name: str, /) -> Never:
    self._raise_attribute_error(name)

def _raise_attribute_error(self, name: str) -> Never:
    # Match the Python 3.12 error messages exactly
    if name == "__name__":
        raise AttributeError("readonly attribute")
    elif name in {"__value__", "__type_params__", "__parameters__", "__module__"}:
        raise AttributeError(
            f"attribute '{name}' of 'typing.TypeAliasType' objects "
            "is not writable"
        )
    else:
        raise AttributeError(
            f"'typing.TypeAliasType' object has no attribute '{name}'"
        )

def __repr__(self) -> str:
    return self.__name__

if sys.version_info < (3, 11):
    def _check_single_param(self, param, recursion=0):
        # Allow [], [int], [int, str], [int, ...], [int, T]
        if param is ...:
            return ...
        if param is None:
            return None
        # Note in <= 3.9 _ConcatenateGenericAlias inherits from list
        if isinstance(param, list) and recursion == 0:
            return [self._check_single_param(arg, recursion+1)
                    for arg in param]
        return typing._type_check(
                param, f'Subscripting {self.__name__} requires a type.'
            )

def _check_parameters(self, parameters):
    if sys.version_info < (3, 11):
        return tuple(
```

```
            self._check_single_param(item)
            for item in parameters
        )
    return tuple(typing._type_check(
                item, f'Subscripting {self.__name__} requires a type.'
            )
            for item in parameters
    )

def __getitem__(self, parameters):
    if not self.__type_params__:
        raise TypeError("Only generic type aliases are subscriptable")
    if not isinstance(parameters, tuple):
        parameters = (parameters,)
    # Using 3.9 here will create problems with Concatenate
    if sys.version_info >= (3, 10):
        return _types.GenericAlias(self, parameters)
    type_vars = _collect_type_vars(parameters)
    parameters = self._check_parameters(parameters)
    alias = _TypeAliasGenericAlias(self, parameters)
    # alias.__parameters__ is not complete if Concatenate is present
    # as it is converted to a list from which no parameters are extracted.
    if alias.__parameters__ != type_vars:
        alias.__parameters__ = type_vars
    return alias

def __reduce__(self):
    return self.__name__

def __init_subclass__(cls, *args, **kwargs):
    raise TypeError(
        "type 'typing_extensions.TypeAliasType' is not an acceptable base type"
    )

# The presence of this method convinces typing._type_check
# that TypeAliasTypes are types.
def __call__(self):
    raise TypeError("Type alias is not callable")

# Breakpoint: https://github.com/python/cpython/pull/21515
if sys.version_info >= (3, 10):
    def __or__(self, right):
        # For forward compatibility with 3.12, reject Unions
        # that are not accepted by the built-in Union.
```

```
            if not _is_unionable(right):
                return NotImplemented
            return typing.Union[self, right]

        def __ror__(self, left):
            if not _is_unionable(left):
                return NotImplemented
            return typing.Union[left, self]


if hasattr(typing, "is_protocol"):
    is_protocol = typing.is_protocol
    get_protocol_members = typing.get_protocol_members
else:
    def is_protocol(tp: type, /) -> bool:
        """Return True if the given type is a Protocol.

        Example::

            >>> from typing_extensions import Protocol, is_protocol
            >>> class P(Protocol):
            ...     def a(self) -> str: ...
            ...     b: int
            >>> is_protocol(P)
            True
            >>> is_protocol(int)
            False
        """
        return (
            isinstance(tp, type)
            and getattr(tp, '_is_protocol', False)
            and tp is not Protocol
            and tp is not typing.Protocol
        )

    def get_protocol_members(tp: type, /) -> typing.FrozenSet[str]:
        """Return the set of members defined in a Protocol.

        Example::

            >>> from typing_extensions import Protocol, get_protocol_members
            >>> class P(Protocol):
            ...     def a(self) -> str: ...
            ...     b: int
```

```
            >>> get_protocol_members(P)
            frozenset({'a', 'b'})

        Raise a TypeError for arguments that are not Protocols.
        """
        if not is_protocol(tp):
            raise TypeError(f'{tp!r} is not a Protocol')
        if hasattr(tp, '__protocol_attrs__'):
            return frozenset(tp.__protocol_attrs__)
        return frozenset(_get_protocol_attrs(tp))


if hasattr(typing, "Doc"):
    Doc = typing.Doc
else:
    class Doc:
        """Define the documentation of a type annotation using ``Annotated``, to be
        used in class attributes, function and method parameters, return values,
        and variables.

        The value should be a positional-only string literal to allow static tools
        like editors and documentation generators to use it.

        This complements docstrings.

        The string value passed is available in the attribute ``documentation``.

        Example::

            >>> from typing_extensions import Annotated, Doc
            >>> def hi(to: Annotated[str, Doc("Who to say hi to")]) -> None: ...
        """
        def __init__(self, documentation: str, /) -> None:
            self.documentation = documentation

        def __repr__(self) -> str:
            return f"Doc({self.documentation!r})"

        def __hash__(self) -> int:
            return hash(self.documentation)

        def __eq__(self, other: object) -> bool:
            if not isinstance(other, Doc):
                return NotImplemented
```

```
            return self.documentation == other.documentation


_CapsuleType = getattr(_types, "CapsuleType", None)

if _CapsuleType is None:
    try:
        import _socket
    except ImportError:
        pass
    else:
        _CAPI = getattr(_socket, "CAPI", None)
        if _CAPI is not None:
            _CapsuleType = type(_CAPI)

if _CapsuleType is not None:
    CapsuleType = _CapsuleType
    __all__.append("CapsuleType")


if sys.version_info >= (3, 14):
    from annotationlib import Format, get_annotations
else:
    # Available since Python 3.14.0a3
    # PR: https://github.com/python/cpython/pull/124415
    class Format(enum.IntEnum):
        VALUE = 1
        VALUE_WITH_FAKE_GLOBALS = 2
        FORWARDREF = 3
        STRING = 4

    # Available since Python 3.14.0a1
    # PR: https://github.com/python/cpython/pull/119891
    def get_annotations(obj, *, globals=None, locals=None, eval_str=False,
                        format=Format.VALUE):
        """Compute the annotations dict for an object.

        obj may be a callable, class, or module.
        Passing in an object of any other type raises TypeError.

        Returns a dict.  get_annotations() returns a new dict every time
        it's called; calling it twice on the same object will return two
        different but equivalent dicts.
```

This is a backport of `inspect.get_annotations`, which has been
in the standard library since Python 3.10. See the standard library
documentation for more:

https://docs.python.org/3/library/inspect.html#inspect.get_annotations

*This backport adds the *format* argument introduced by PEP 649. The*
three formats supported are:
** VALUE: the annotations are returned as-is. This is the default and*
it is compatible with the behavior on previous Python versions.
** FORWARDREF: return annotations as-is if possible, but replace any*
undefined names with ForwardRef objects. The implementation proposed by
PEP 649 relies on language changes that cannot be backported; the
typing-extensions implementation simply returns the same result as VALUE.
** STRING: return annotations as strings, in a format close to the original*
source. Again, this behavior cannot be replicated directly in a backport.
As an approximation, typing-extensions retrieves the annotations under
VALUE semantics and then stringifies them.

The purpose of this backport is to allow users who would like to use
FORWARDREF or STRING semantics once PEP 649 is implemented, but who also
want to support earlier Python versions, to simply write:

typing_extensions.get_annotations(obj, format=Format.FORWARDREF)

"""
```
format = Format(format)
if format is Format.VALUE_WITH_FAKE_GLOBALS:
    raise ValueError(
        "The VALUE_WITH_FAKE_GLOBALS format is for internal use only"
    )

if eval_str and format is not Format.VALUE:
    raise ValueError("eval_str=True is only supported with format=Format.VALUE")

if isinstance(obj, type):
    # class
    obj_dict = getattr(obj, '__dict__', None)
    if obj_dict and hasattr(obj_dict, 'get'):
        ann = obj_dict.get('__annotations__', None)
        if isinstance(ann, _types.GetSetDescriptorType):
            ann = None
    else:
        ann = None
```

```
    obj_globals = None
    module_name = getattr(obj, '__module__', None)
    if module_name:
        module = sys.modules.get(module_name, None)
        if module:
            obj_globals = getattr(module, '__dict__', None)
    obj_locals = dict(vars(obj))
    unwrap = obj
elif isinstance(obj, _types.ModuleType):
    # module
    ann = getattr(obj, '__annotations__', None)
    obj_globals = obj.__dict__
    obj_locals = None
    unwrap = None
elif callable(obj):
    # this includes types.Function, types.BuiltinFunctionType,
    # types.BuiltinMethodType, functools.partial, functools.singledispatch,
    # "class funclike" from Lib/test/test_inspect... on and on it goes.
    ann = getattr(obj, '__annotations__', None)
    obj_globals = getattr(obj, '__globals__', None)
    obj_locals = None
    unwrap = obj
elif hasattr(obj, '__annotations__'):
    ann = obj.__annotations__
    obj_globals = obj_locals = unwrap = None
else:
    raise TypeError(f"{obj!r} is not a module, class, or callable.")

if ann is None:
    return {}

if not isinstance(ann, dict):
    raise ValueError(f"{obj!r}.__annotations__ is neither a dict nor None")

if not ann:
    return {}

if not eval_str:
    if format is Format.STRING:
        return {
            key: value if isinstance(value, str) else typing._type_repr(value)
            for key, value in ann.items()
        }
```

```
            return dict(ann)

        if unwrap is not None:
            while True:
                if hasattr(unwrap, '__wrapped__'):
                    unwrap = unwrap.__wrapped__
                    continue
                if isinstance(unwrap, functools.partial):
                    unwrap = unwrap.func
                    continue
                break
            if hasattr(unwrap, "__globals__"):
                obj_globals = unwrap.__globals__

        if globals is None:
            globals = obj_globals
        if locals is None:
            locals = obj_locals or {}

        # "Inject" type parameters into the local namespace
        # (unless they are shadowed by assignments *in* the local namespace),
        # as a way of emulating annotation scopes when calling `eval()`
        if type_params := getattr(obj, "__type_params__", ()):
            locals = {param.__name__: param for param in type_params} | locals

        return_value = {key:
            value if not isinstance(value, str) else eval(value, globals, locals)
            for key, value in ann.items() }
        return return_value


if hasattr(typing, "evaluate_forward_ref"):
    evaluate_forward_ref = typing.evaluate_forward_ref
else:
    # Implements annotationlib.ForwardRef.evaluate
    def _eval_with_owner(
        forward_ref, *, owner=None, globals=None, locals=None, type_params=None
    ):
        if forward_ref.__forward_evaluated__:
            return forward_ref.__forward_value__
        if getattr(forward_ref, "__cell__", None) is not None:
            try:
                value = forward_ref.__cell__.cell_contents
            except ValueError:
```

```
            pass
        else:
            forward_ref.__forward_evaluated__ = True
            forward_ref.__forward_value__ = value
            return value
    if owner is None:
        owner = getattr(forward_ref, "__owner__", None)

    if (
        globals is None
        and getattr(forward_ref, "__forward_module__", None) is not None
    ):
        globals = getattr(
            sys.modules.get(forward_ref.__forward_module__, None), "__dict__", None
        )
    if globals is None:
        globals = getattr(forward_ref, "__globals__", None)
    if globals is None:
        if isinstance(owner, type):
            module_name = getattr(owner, "__module__", None)
            if module_name:
                module = sys.modules.get(module_name, None)
                if module:
                    globals = getattr(module, "__dict__", None)
        elif isinstance(owner, _types.ModuleType):
            globals = getattr(owner, "__dict__", None)
        elif callable(owner):
            globals = getattr(owner, "__globals__", None)

    # If we pass None to eval() below, the globals of this module are used.
    if globals is None:
        globals = {}

    if locals is None:
        locals = {}
        if isinstance(owner, type):
            locals.update(vars(owner))

    if type_params is None and owner is not None:
        # "Inject" type parameters into the local namespace
        # (unless they are shadowed by assignments *in* the local namespace),
        # as a way of emulating annotation scopes when calling `eval()`
        type_params = getattr(owner, "__type_params__", None)
```

```
    # Type parameters exist in their own scope, which is logically
    # between the locals and the globals. We simulate this by adding
    # them to the globals.
    if type_params is not None:
        globals = dict(globals)
        for param in type_params:
            globals[param.__name__] = param

    arg = forward_ref.__forward_arg__
    if arg.isidentifier() and not keyword.iskeyword(arg):
        if arg in locals:
            value = locals[arg]
        elif arg in globals:
            value = globals[arg]
        elif hasattr(builtins, arg):
            return getattr(builtins, arg)
        else:
            raise NameError(arg)
    else:
        code = forward_ref.__forward_code__
        value = eval(code, globals, locals)
    forward_ref.__forward_evaluated__ = True
    forward_ref.__forward_value__ = value
    return value

def evaluate_forward_ref(
    forward_ref,
    *,
    owner=None,
    globals=None,
    locals=None,
    type_params=None,
    format=None,
    _recursive_guard=frozenset(),
):
    """Evaluate a forward reference as a type hint.

    This is similar to calling the ForwardRef.evaluate() method,
    but unlike that method, evaluate_forward_ref() also:

    * Recursively evaluates forward references nested within the type hint.
    * Rejects certain objects that are not valid type hints.
    * Replaces type hints that evaluate to None with types.NoneType.
    * Supports the *FORWARDREF* and *STRING* formats.
```

**forward_ref* must be an instance of ForwardRef. *owner*, if given, should be the object that holds the annotations that the forward reference derived from, such as a module, class object, or function. It is used to infer the namespaces to use for looking up names. *globals* and *locals* can also be explicitly given to provide the global and local namespaces. *type_params* is a tuple of type parameters that are in scope when evaluating the forward reference. This parameter must be provided (though it may be an empty tuple) if *owner* is not given and the forward reference does not already have an owner set. *format* specifies the format of the annotation and is a member of the annotationlib.Format enum.*

"""
```
if format == Format.STRING:
    return forward_ref.__forward_arg__
if forward_ref.__forward_arg__ in _recursive_guard:
    return forward_ref

# Evaluate the forward reference
try:
    value = _eval_with_owner(
        forward_ref,
        owner=owner,
        globals=globals,
        locals=locals,
        type_params=type_params,
    )
except NameError:
    if format == Format.FORWARDREF:
        return forward_ref
    else:
        raise

if isinstance(value, str):
    value = ForwardRef(value)

# Recursively evaluate the type
if isinstance(value, ForwardRef):
    if getattr(value, "__forward_module__", True) is not None:
        globals = None
    return evaluate_forward_ref(
        value,
        globals=globals,
        locals=locals,
```

```
            type_params=type_params, owner=owner,
            _recursive_guard=_recursive_guard, format=format
        )
    if sys.version_info < (3, 12, 5) and type_params:
        # Make use of type_params
        locals = dict(locals) if locals else {}
        for tvar in type_params:
            if tvar.__name__ not in locals:  # lets not overwrite something present
                locals[tvar.__name__] = tvar
    if sys.version_info < (3, 12, 5):
        return typing._eval_type(
            value,
            globals,
            locals,
            recursive_guard=_recursive_guard | {forward_ref.__forward_arg__},
        )
    else:
        return typing._eval_type(
            value,
            globals,
            locals,
            type_params,
            recursive_guard=_recursive_guard | {forward_ref.__forward_arg__},
        )


class Sentinel:
    """Create a unique sentinel object.

    *name* should be the name of the variable to which the return value shall be assigned.

    *repr*, if supplied, will be used for the repr of the sentinel object.
    If not provided, "<name>" will be used.
    """

    def __init__(
        self,
        name: str,
        repr: typing.Optional[str] = None,
    ):
        self._name = name
        self._repr = repr if repr is not None else f'<{name}>'

    def __repr__(self):
```

```
        return self._repr

    if sys.version_info < (3, 11):
        # The presence of this method convinces typing._type_check
        # that Sentinels are types.
        def __call__(self, *args, **kwargs):
            raise TypeError(f"{type(self).__name__!r} object is not callable")

    # Breakpoint: https://github.com/python/cpython/pull/21515
    if sys.version_info >= (3, 10):
        def __or__(self, other):
            return typing.Union[self, other]

        def __ror__(self, other):
            return typing.Union[other, self]

    def __getstate__(self):
        raise TypeError(f"Cannot pickle {type(self).__name__!r} object")


if sys.version_info >= (3, 14, 0, "beta"):
    type_repr = annotationlib.type_repr
else:
    def type_repr(value):
        """Convert a Python value to a format suitable for use with the STRING format.

        This is intended as a helper for tools that support the STRING format but do
        not have access to the code that originally produced the annotations. It uses
        repr() for most objects.

        """
        if isinstance(value, (type, _types.FunctionType, _types.BuiltinFunctionType)):
            if value.__module__ == "builtins":
                return value.__qualname__
            return f"{value.__module__}.{value.__qualname__}"
        if value is ...:
            return "..."
        return repr(value)


# Aliases for items that are in typing in all supported versions.
# We use hasattr() checks so this library will continue to import on
# future versions of Python that may remove these names.
_typing_names = [
```

```
    "AbstractSet",
    "AnyStr",
    "BinaryIO",
    "Callable",
    "Collection",
    "Container",
    "Dict",
    "FrozenSet",
    "Hashable",
    "IO",
    "ItemsView",
    "Iterable",
    "Iterator",
    "KeysView",
    "List",
    "Mapping",
    "MappingView",
    "Match",
    "MutableMapping",
    "MutableSequence",
    "MutableSet",
    "Optional",
    "Pattern",
    "Reversible",
    "Sequence",
    "Set",
    "Sized",
    "TextIO",
    "Tuple",
    "Union",
    "ValuesView",
    "cast",
    "no_type_check",
    "no_type_check_decorator",
    # This is private, but it was defined by typing_extensions for a long time
    # and some users rely on it.
    "_AnnotatedAlias",
]
globals().update(
    {name: getattr(typing, name) for name in _typing_names if hasattr(typing, name)}
)
# These are defined unconditionally because they are used in
# typing-extensions itself.
Generic = typing.Generic
```

```
ForwardRef = typing.ForwardRef
Annotated = typing.Annotated
```

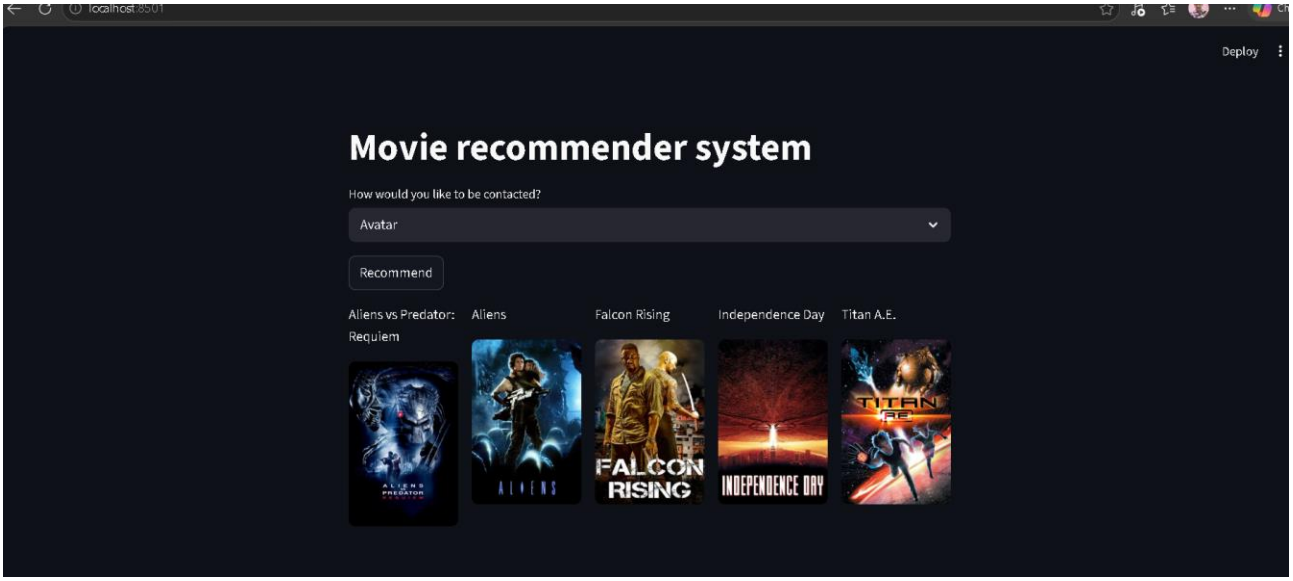
❖❖❖❖❖ **End of Chapter** ❖❖❖❖❖

CHAPTER 12: OUTPUT SCREENS

Output Example

User Input:
Movie Name: Avatar
Recommended Movies:

1. Guardians of the Galaxy
2. Interstellar
3. John Carter
4. Star Trek
5. The Martian



❖❖❖❖❖ **End of Chapter** ❖❖❖❖❖

CHAPTER 13: ADVANTAGES AND LIMITATIONS

Advantages

- Saves user time.
- Provides personalized recommendations.
- Easy to use.

- Fast recommendation process.

Limitations

- Depends on dataset quality.
- Cannot recommend movies outside the dataset.
- Accuracy may vary.

❖❖❖❖❖ **End of Chapter** ❖❖❖❖❖

CHAPTER 14: FUTURE SCOPE

Future improvements can include:

- Developing a web application.
- Adding user login system.
- Using Deep Learning algorithms.
- Integrating real-time movie data.
- Creating mobile applications.
- ❖❖❖❖❖ **End of Chapter** ❖❖❖❖❖

CHAPTER 15: CONCLUSION

The Movie Recommendation System successfully recommends movies based on similarity and user preferences using Machine Learning techniques.

This project helped in understanding recommendation algorithms, data preprocessing, feature extraction, and similarity calculations.

The system improves user experience and demonstrates practical implementation of Machine Learning concepts.

❖❖❖❖❖ **End of Chapter** ❖❖❖❖❖

CHAPTER 16: REFERENCES

1. Python Official Documentation
2. Scikit-learn Documentation
3. Pandas Documentation
4. Machine Learning Tutorials
5. Research Papers on Recommendation Systems

Working demo: Streamlit

Full github coed: https://github.com/shuvammandal516-arch/Movie-recommendation-system

Dataset Link: https://www.kaggle.com/tmdb/tmdb-movi...

❖❖❖❖❖ **End of Chapter** ❖❖❖❖❖

➢ *APPENDIX:-*

A. Source Code:

```
#%%

import numpy as np

import pandas as pd

#%%

movies = pd.read_csv('tmdb_5000_movies.csv')

credits = pd.read_csv('tmdb_5000_credits.csv')

#%%

movies = movies.merge(credits,on='title')

#%%

movies.head(1)

#%%

movies.info()

#%%

movies = movies[['movie_id' , 'title' , 'overview' , 'genres' ,
    'keywords' , 'cast' , 'crew']]

#%%

movies.head(1)

#%%

movies.isnull().sum()

#%%

movies.dropna(inplace=True)
```

```
#%%
movies.duplicated().sum()
#%%
movies.iloc[0].genres
#%%
import ast
#%%
def convert(obj):
    L = []
    for i in ast.literal_eval(obj):
        L.append(i['name'])
    return L
#%%
movies['genres'] = movies['genres'].apply(convert)
movies.head(1)
#%%
movies['keywords'] = movies['keywords'].apply(convert)
movies.head(1)
#%%
def convert3(obj):
    L = []
    counter = 0
    for i in ast.literal_eval(obj):
        if counter != 3:
            L.append(i['name'])
```

```
            counter+=1
        else:
            break
    return L
#%%
movies['cast'] = movies['cast'].apply(convert3)
movies.head()
#%%
def fetch_director(text):
    L = []
    for i in ast.literal_eval(text):
        if i['job'] == 'Director':
            L.append(i['name'])
    return L
#%%
movies['crew'] = movies['crew'].apply(fetch_director)
movies.head()
#%%
movies['overview'] = movies['overview'].apply(lambda
    x:x.split())
#%%
movies.head(1)
#%%
movies['genres'] = movies['genres'].apply(lambda x: [i.replace("
    ","") for i in x])
```

```
movies['keywords'] = movies['keywords'].apply(lambda x:
    [i.replace(" ","") for i in x])

movies['cast'] = movies['cast'].apply(lambda x: [i.replace(" ","")
    for i in x])

movies['crew'] = movies['crew'].apply(lambda x: [i.replace(" ","")
    for i in x])

#%%

movies.head(1)

#%%

movies['tags'] = movies['overview'] + movies['genres'] +
    movies['keywords'] + movies['cast'] + movies['crew']

#%%

movies.head(1)

#%%

new_df = movies[['movie_id' , 'title' , 'tags']]

#%%

new_df.head()

#%%

new_df['tags'] = new_df['tags'].apply(lambda x: " ".join(x))


#%%

new_df.head()

#%%

new_df['tags'][0]

#%%

new_df['tags'] = new_df['tags'].apply(lambda x:x.lower())
```

```
#%%

new_df.head()

#%%

!pip install nltk

#%%

import nltk

#%%

from nltk.stem.porter import PorterStemmer

ps =PorterStemmer()

#%%

def stem(text):

    y = []

    for i in text.split():

        y.append(ps.stem(i))

    return " ".join(y)

#%%

new_df['tags'] = new_df['tags'].apply(stem)

#%%

from sklearn.feature_extraction.text import CountVectorizer

cv = CountVectorizer(max_features =
     5000,stop_words='english')

#%%

vectors = cv.fit_transform(new_df['tags']).toarray()

#%%

vectors
```

```
#%%

from sklearn.metrics.pairwise import cosine_similarity

#%%

similarity = cosine_similarity(vectors)

#%%

similarity[1]

#%%

def recommend(movie):

    movie_index = new_df[new_df['title'] ==movie].index[0]

    distances = similarity[movie_index]

    movies_list =
     sorted(list(enumerate(distances)),reverse=True,key=lambda
     x:x[1])[1:6]


    for i in movies_list:

        print(new_df.iloc[i[0]].title)


#%%

recommend('Avatar')

#%%

import pickle

#%%

pickle.dump(new_df,open('movies.pkl','wb'))

#%%

new_df['title'].values
```

```
#%%

pickle.dump(new_df.to_dict(),open('movie_dict.pkl','wb'))

#%%

pickle.dump(similarity,open('similarity.pkl','wb'))

    #%%
```

B.app.py

```
import streamlit as st
    import pickle
    import pandas as pd
    import requests

    def fetch_poster(movie_id):
        response =
    requests.get("https://api.themoviedb.org/3/movie/{}?api_ke
    y=8265bd1679663a7ea12ac168da84d2e8&language=en-
    US".format(movie_id))
        data = response.json()
        return "https://image.tmdb.org/t/p/w500/" +
    data['poster_path']

    def recommend(movie):
        movie_index = movies[movies['title'] ==movie].index[0]
        distances = similarity[movie_index]
        movies_list =
    sorted(list(enumerate(distances)),reverse=True,key=lambda
    x:x[1])[1:6]

        recommended_movies = []
        recommended_movies_posters = []
        for i in movies_list:
            movie_id = movies.iloc[i[0]].movie_id
```

```
        recommended_movies.append(movies.iloc[i[0]].title)
        # fetch poster from api

recommended_movies_posters.append(fetch_poster(movie_
id))
    return
recommended_movies,recommended_movies_posters


movies_dict = pickle.load(open("movie_dict.pkl", "rb"))
movies = pd.DataFrame(movies_dict)

similarity = pickle.load(open("similarity.pkl", "rb"))


st.title('Movie recommender system')
selected_movie_name = st.selectbox(
    'How would you like to be contacted?',
    movies['title'].values
)


if st.button('Recommend'):
   names,posters = recommend(selected_movie_name)

   col1, col2, col3, col4, col5 = st.columns(5)
   with col1:
      st.text(names[0])
      st.image(posters[0])
   with col2:
      st.text(names[1])
      st.image(posters[1])

   with col3:
      st.text(names[2])
      st.image(posters[2])
   with col4:
```

```
    st.text(names[3])
    st.image(posters[3])
with col5:
    st.text(names[4])
    st.image(posters[4])
```

THANK YOU !